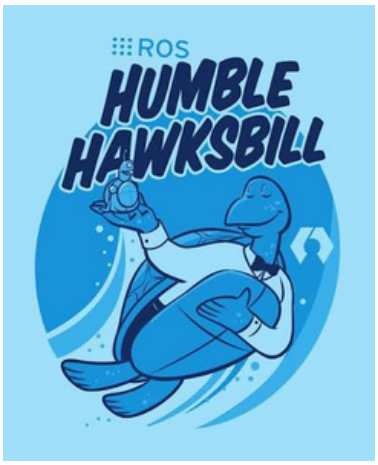
Université Abdelmalek Essaâdi
Ecole Nationale des Sciences Appliquées
Tanger

جامعة عبد المالك السعدي
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵄⴰⴱⴷⵍⵎⴰⵍⵉⴽ ⵙⵙⴰⵄⴷⵉ
Université Abdelmalek Essaadi

Projet – Autonomous Navigation for a Mobile Robot Using ROS 2

Rapport de Projet Collectif



Présentés par :

KCHIT Hajar
MAIMOUN - CHEKRI Mohammed
RAVELOHARISOA Mickaella Fandresena
ANDRIASAHY Tsikomia Scidelo
LAGHOUIYEL Youssef

Sous la supervision de :

Pr. Khadija EL HAMIDI

Professeure à l'ENSA de Tanger

Année universitaire 2025-2026

REMERCIEMENTS

« Seul on va plus vite, ensemble on va plus loin. »

Nous tenons tout d'abord à exprimer notre plus profonde gratitude à notre encadrante de projet, **Pr. Khadija EL HAMIDI**. Ses précieux conseils, sa rigueur scientifique et son soutien constant tout au long de cette année ont été des piliers fondamentaux pour la réussite de la conception de notre robot mobile autonome. Nous la remercions pour le temps qu'elle nous a consacré et pour la confiance qu'elle nous a accordée face aux défis techniques rencontrés.

Nos remerciements s'adressent également à l'ensemble du corps professoral et à la direction de la formation **GSEA2**. Leurs enseignements de qualité nous ont fourni le bagage technique et théorique indispensable pour aborder sereinement un domaine aussi complexe que l'architecture ROS 2 et la robotique d'exploration.

Enfin, nous tenons à souligner l'incroyable synergie qui a régné au sein de notre équipe. La réalisation de ce système complet, de sa modélisation physique sous Gazebo jusqu'à l'implémentation de la navigation autonome (Nav2), est le fruit d'un travail collaboratif intense entre le pôle **Navigation** et le pôle **Perception**.

Merci à chacun des membres du groupe : Kchit, Maimoun-Chekri, Raveloharisoa, Laghouiyel et Andriasahy, pour son dévouement, ses longues sessions de débogage et son esprit d'entraide permanent.

L'équipe GSEA2

RÉSUMÉS / ABSTRACTS

Résumé (Français)

Ce rapport détaille la conception, la modélisation et la programmation d'un robot mobile autonome à entraînement différentiel, réalisé dans le cadre du projet académique GSEA2. L'architecture logicielle repose sur le middleware ROS 2 (Humble) et s'articule autour de nœuds modulaires. Le développement collaboratif a été organisé selon la méthode Agile via GitHub. Le robot a été modélisé en URDF/Xacro et simulé sous Gazebo 3D, intégrant des capteurs virtuels (LiDAR, IMU) pour la perception environnementale. Le premier sprint a permis d'établir une téléopération manuelle sécurisée. Le second sprint s'est concentré sur la cartographie spatiale simultanée (SLAM Toolbox) et la configuration avancée de la navigation autonome (Nav2, Costmaps), permettant au robot de détecter et d'éviter les obstacles de manière dynamique.

Mots-clés : ROS 2, Gazebo, Robotique Autonome, SLAM, Nav2, URDF, Costmap.

Abstract (English)

This report details the design, modeling, and programming of an autonomous differential drive mobile robot, developed as part of the GSEA2 academic project. The software architecture is based on the ROS 2 (Humble) middleware and structured around modular nodes. Collaborative development was organized using Agile methodology via GitHub. The robot was modeled using URDF/Xacro and simulated in Gazebo 3D, integrating virtual sensors (LiDAR, IMU) for environmental perception. The first sprint established secure manual teleoperation. The second sprint focused on simultaneous spatial mapping (SLAM Toolbox) and the advanced configuration of autonomous navigation (Nav2, Costmaps), allowing the robot to dynamically detect and avoid obstacles.

Keywords : ROS 2, Gazebo, Autonomous Robotics, SLAM, Nav2, URDF, Costmap.

(Arabe)

يفصل هذا التقرير تصميم ونمذجة وبرمجة روبوت محمول مستقل ذي دفع تفاضلي، تم تطويره كجزء من مشروع GSEA2 الأكاديمي. تعتمد بنية البرمجيات على نظام التشغيل ROS 2 (Humble) وتتمحور حول عقد معيارية. تم تنظيم التطوير التعاوني باستخدام GitHub. تمت نمذجة الروبوت باستخدام URDF/Xacro ومحاكاته في بيئة Gazebo 3D، مع دمج أجهزة استشعار افتراضية (LiDAR, IMU) للإدراك البيئي. أسس السبرينت الأول تحكمًا يدويًا آمنًا عن بُعد. بينما ركز السبرينت الثاني على رسم الخرائط المكانية المتزامنة (SLAM Toolbox) والتكوين المتقدم للملاحة المستقلة (Nav2, Costmaps)، مما يسمح للروبوت باكتشاف العقبات وتجنبها ديناميكيًا.
الكلمات المفتاحية: ROS 2, Gazebo, الروبوتات المستقلة, SLAM, Nav2, URDF, Costmap.

Table des matières

Remerciements 2

Résumés / Abstracts 3

Table des figures 9

Table des tableaux 10

1 Introduction Générale et Contexte 11
- 1.1 Contexte de la Robotique Mobile Moderne . . . 11
- 1.2 Démystification des Concepts Logiciels Clés . . . 11
  - 1.2.1 Qu'est-ce qu'un Framework ? . . . 11
  - 1.2.2 Qu'est-ce qu'un Middleware ? . . . 12
- 1.3 Présentation du Projet GSEA2 et de ses Outils . . . 13
  - 1.3.1 Modélisation Physique : URDF et Xacro . . . 13
  - 1.3.2 Simulation 3D : Gazebo . . . 13
  - 1.3.3 Les Capteurs Virtuels : LiDAR 2D et IMU . . . 14
  - 1.3.4 Intelligence et Autonomie : Nav2 et SLAM . . . 14
- 1.4 Le Workflow du Système . . . 15
- 1.5 Organisation de l'Équipe et Sprints . . . 16
- 1.6 Structure du Rapport . . . 16

2 Environnement de Travail et Méthodologie Collaborative 18
- 2.1 Infrastructure Système et Choix de Déploiement . . . 18
  - 2.1.1 Dilemme Technique : Machine Virtuelle vs Dual Boot . . . 18
  - 2.1.2 Gestion du Rendu Graphique sur VirtualBox . . . 19
- 2.2 Procédure d'Installation Étape par Étape . . . 19
  - 2.2.1 Configuration des Locales et des Dépôts Apt . . . 19
  - 2.2.2 Installation de ROS 2 Humble et des Packages d'Intelligence . . . 20
  - 2.2.3 Persistance de l'Environnement (Sourcing) . . . 20
- 2.3 L'Espace de Travail ROS 2 (Workspace) . . . 21
  - 2.3.1 L'arborescence standard d'un espace de travail . . . 21
  - 2.3.2 Création et Compilation d'un Package Python . . . 22
  - 2.3.3 Analyse de l'Arborescence du Package Autonome . . . 22
- 2.4 Gestion de Version et Synchronisation de l'Équipe . . . 24
  - 2.4.1 Le Workflow Git/GitHub (Logigramme) . . . 24
  - 2.4.2 Standardisation des Commandes (Terminal vs Éditeur) . . . 25
- 2.5 Approche Agile : Le Découpage en Sprints . . . 26

3 Fondements Logiques et Modélisation Physique 27

3.1 Le Graphe de Calcul : L'écosystème ROS 2 27
3.2 Communication et Échange de Données 28
3.2.1 Les Topics : Flux de données continu 28
3.2.2 Les Services : Requête et Réponse 28
3.2.3 Illustration des Communications 29
3.3 Modélisation Physique : De l'URDF à Xacro 29
3.3.1 La puissance de Xacro 30
3.3.2 Architecture Modulaire du Modèle Xacro 30
3.3.3 Nomenclature des Composants Physiques 31
3.3.4 L'Interface Physique-Logique : Les Plugins Gazebo 32
3.4 Cinématique Différentielle (Differential Drive) 32

4 Pilotage, Perception et Cartographie Simultanée (SLAM) 34
4.1 Le Système de Téléopération Clavier (`robot_teleop`) 34
4.1.1 L'Architecture d'Exécution et Isolation des Processus 34
4.1.2 Analyse Algorithmique du Script `teleop.py` 34
4.2 Perception Sensorielle : Les Capteurs Virtuels Embarqués 36
4.2.1 Le LiDAR 2D (Laser Scan) 36
4.2.2 La Centrale Inertielle (IMU) 36
4.3 Cartographie SLAM : Implémentation et Algorithmique 37
4.3.1 Configuration Initiale et Fichiers Paramétriques 37
4.3.2 Fonctionnement Interne : Le Graphe d'Optimisation 37
4.4 Fondements Mathématiques de la Cartographie SLAM 38
4.4.1 Principe du Graph-SLAM et Scan Matching 38
4.4.2 Optimisation du Graphe via Ceres Solver 39
4.5 Génération Pratique de la Carte (Téléopération) 39
4.5.1 Protocole d'Exploration Spatiale 39
4.5.2 Publication de la Carte et Correction Globale 40
4.5.3 Sauvegarde et Exportation de l'Occupancy Grid 40
4.5.4 Visualisation et Gestion des Erreurs de Repères (TF) 41
4.5.5 Configuration Manuelle de l'Interface RViz2 41
4.5.6 Génération Finale et Sauvegarde de la Grille d'Occupation 42

5 Navigation Autonome et Planification de Trajectoires 44
5.1 Introduction au Framework Nav2 44
5.2 Théorie des Cartes de Coût (Costmaps) 44
5.2.1 La Global Costmap (Carte de coût globale) 45
5.2.2 La Local Costmap (Carte de coût locale) 45
5.3 Configuration Stratégique des Paramètres (YAML) 45
5.3.1 Analyse de l'Empreinte : `robot_radius` 45
5.3.2 La Bulle de Sécurité : `inflation_radius` 45
5.3.3 Synchronisation Temporelle : `use_sim_time` 46
5.4 La Planification : Global vs Local Planner 46
5.4.1 Le Planificateur Global (`NavfnPlanner`) 46
5.4.2 Le Contrôleur Local (`DWBLocalPlanner`) 46
5.5 Localisation Probabiliste : L'Algorithme AMCL 47
5.5.1 La Calibration de la Pose Initiale (2D Pose Estimate) 47
5.6 Exécution d'une Requête de Navigation (Actions ROS 2) 47

5.7 Validation et Tests d'Évitement d'Obstacles 48
5.7.1 Scénarios de Simulation Validés et Analyse Visuelle 49
5.7.2 Bilan de Synthèse du Pôle Navigation 51

6 Conclusion Générale et Perspectives **52**
6.1 Bilan Technique et Synthèse du Projet 52
6.2 Difficultés Résolues et Compétences Acquises 52
6.3 Perspectives de Développement Futur 53
6.3.1 Migration vers une Plateforme Physique Réelle 53
6.3.2 Intégration de la Vision par Ordinateur (Visual SLAM) 53
6.3.3 Évitement d'Obstacles par Apprentissage par Renforcement (IA) . 53
6.3.4 Robotique Collaborative (Flotte de Robots) 53

ANNEXES **55**

A Guide de Reproductibilité et Dépôt GitHub **56**

B Extraits de Code : Configuration Nav2 **57**

C Script de Lancement Principal (Launch File) **58**

D Script de Lancement de la Navigation (Nav2) **60**

Références Bibliographiques **62**
Books 62
Articles 62
Chapitre de livres 62
Cours et Ouvrages Académiques 62
Autres 62

Table des figures

1.1 Middleware . . . 12
1.2 Environnement de simulation du robot sous Gazebo 3D . . . 13
1.3 Scan laser par le LiDAR 2D . . . 14
1.4 Représentation des axes de l'IMU . . . 14
1.5 Visualisation dans RViz2 de la carte générée (SLAM) et de la trajectoire (Nav2) . . . 15
1.6 Workflow de la boucle de rétroaction du robot autonome . . . 15
1.7 Feuille de route globale (Roadmap) du développement du robot mobile sous ROS 2 . . . 16

2.1 Arborescence standard d'un workspace ROS 2 . . . 22
2.2 Représentation visuelle de l'arborescence du package ROS 2 de notre projet 24
2.3 Workflow de synchronisation collaborative avec Git et GitHub . . . 25

3.1 Architecture logicielle globale : Nœuds ROS 2 et échanges via les Topics . 28
3.2 Modèle Publisher / Subscriber (Topic)[5] . . . 29
3.3 Modèle Client / Serveur (Service)[5] . . . 29
3.4 Architecture Modulaire du Modèle Xacro . . . 31
3.5 Robot à entraînement différentiel dans Gazebo . . . 33

4.1 Téléopération Clavier . . . 35
4.2 Processus algorithmique interne : du capteur brut à la grille d'occupation (Karto SLAM) . . . 38
4.3 Représentation topologique d'un Graph-SLAM avec fermeture de boucle . 39
4.4 Paramétrage de SLAM Toolbox . . . 41
4.5 Configuration du module RobotModel . . . 42
4.6 Activation des repères spatiaux (TF) . . . 42
4.7 Paramétrage du flux LaserScan . . . 42
4.8 Ajout de la grille d'occupation (Map) . . . 42
4.9 Rendu global de l'environnement virtuel dans l'interface RViz2 . . . 43

5.1 Workflow détaillé de la stack de navigation Nav2 . . . 46
5.2 Flux de travail : Test et simulation du robot . . . 48
5.3 Monde de démonstration initial . . . 49
5.4 Monde labyrinthique de validation finale . . . 49
5.5 Évolution des environnements de simulation 3D sous Gazebo . . . 49
5.6 Vue physique du robot dans Gazebo . . . 50
5.7 Visualisation du chemin global dans RViz2 . . . 50
5.8 Corrélation synchrone de la navigation : Physique vs Logique . . . 50
5.9 Analyse de l'empreinte circulaire du modèle de base sous RViz2 . . . 50
5.10 Planification locale en couloir étroit . . . 51

5.11 Contournement réussi d'un obstacle non convexe 51
5.12 Comportement dynamique du nuage de particules AMCL et de la trajectoire locale 51

Liste des tableaux

1.2 Répartition des rôles et organisation de l'équipe GSEA2 16
1.1 Liste exhaustive des outils et technologies utilisés pour le robot mobile autonome 17

2.1 Comparatif technique entre le déploiement par Machine Virtuelle et Dual Boot 19
2.2 Traduction des commandes de versionnage : Théorie vs Pratique 25

3.1 Comparaison technique entre les Topics et les Services ROS 2 29
3.2 Inventaire architectural des composants physiques du robot 32

CHAPITRE 1

INTRODUCTION GÉNÉRALE ET CONTEXTE

1.1 Contexte de la Robotique Mobile Moderne

Au XXIe siècle, la robotique a franchi un cap majeur en passant de simples automates programmés à des systèmes **hautement autonomes et intelligents**. La robotique mobile moderne ne se contente plus d'exécuter des mouvements répétitifs ; elle doit **percevoir son environnement**, **analyser des situations complexes** et **prendre des décisions en temps réel**.

Cependant, développer un tel niveau d'intelligence à partir de zéro (from scratch) est une tâche titanesque. C'est pour répondre à ce défi que la communauté scientifique et industrielle s'est tournée vers des outils standardisés et open-source.

1.2 Démystification des Concepts Logiciels Clés

Pour bien appréhender l'architecture de notre robot, il est impératif de définir et de distinguer les briques logicielles fondamentales sur lesquelles repose **ROS 2**.

1.2.1 Qu'est-ce qu'un Framework ?

Un **Framework** (ou "cadre de travail" en français) est une structure logicielle globale qui fournit un squelette d'application prêt à l'emploi. Contrairement à un simple outil, le framework dicte la façon dont l'architecture du projet doit être construite. La caractéristique majeure d'un framework est *l'inversion de contrôle* : ce n'est pas votre code qui appelle le framework, c'est le framework qui appelle votre code au bon moment.

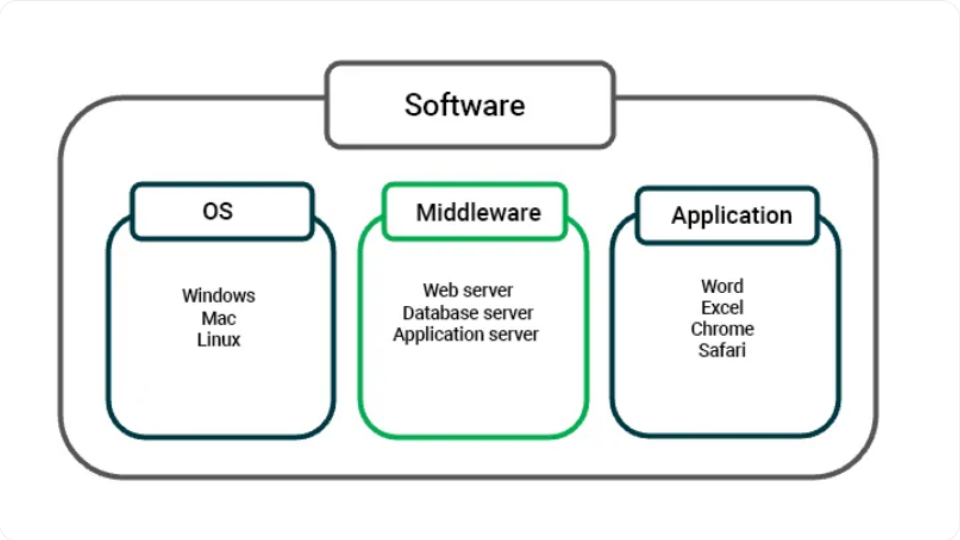
- **La Bibliothèque (Library) :** C'est une simple boîte à outils (ex : `NumPy` en Python ou `Math.h` en C). Vous piochez une fonction mathématique quand vous en avez besoin. Vous gardez le contrôle total.
- **L'API (Application Programming Interface) :** C'est le "menu du restaurant". Elle définit comment deux logiciels peuvent communiquer (les requêtes possibles et les réponses attendues), sans fournir le code interne.

- **Le SDK (Software Development Kit) :** C'est la "caisse à outils complète du constructeur". Il contient souvent un framework, des bibliothèques, un compilateur et une API (ex : Android SDK pour faire des applications mobiles).

Exemple : ROS est souvent qualifié de framework robotique car il impose de structurer son code sous forme de "Nœuds" (Nodes) avec une logique précise, tout comme **Django** est un framework pour le web.

1.2.2 Qu'est-ce qu'un Middleware ?

Un **Middleware** (ou "intergiciel") est un logiciel qui se situe *au milieu*, entre le système d'exploitation et les applications. Son rôle est de masquer la complexité du réseau. Imaginez-le comme un service de traduction et de livraison postal ultra-rapide dans une entreprise multinationale : il s'assure que le "Nœud Caméra" puisse envoyer ses images au "Nœud Navigation" de manière instantanée, même s'ils sont codés dans des langages différents (C++ et Python).



[6]

FIGURE 1.1 – Middleware

- **Le Système d'Exploitation (OS) :** (ex : Ubuntu, Windows). Il gère le matériel brut (mémoire, processeur, carte wifi).
- **Le Message Broker (Courtier de messages) :** (ex : RabbitMQ, MQTT). C'est un type de middleware très simple qui se contente de faire transiter des messages d'un point A à un point B.
- **Le DDS (Data Distribution Service) :** C'est le middleware spécifique utilisé par **ROS 2**. C'est un standard industriel ultra-robuste, pensé pour le temps réel (utilisé dans l'aéronautique et la défense).

Différence clé : L'OS gère l'ordinateur physique. Le Broker gère des messages simples. Le Middleware DDS de ROS 2 fait bien plus : il gère des messages complexes, des flux vidéos, et découvre automatiquement les autres robots sur le réseau sans serveur central.

1.3 Présentation du Projet GSEA2 et de ses Outils

Notre projet , encadré par **Pr. Khadija EL HAMIDI**, a pour objectif la conception et la programmation d'un **Robot Mobile Autonome** à entraînement différentiel (Differential Drive). Pour ce faire, nous avons combiné plusieurs technologies de pointe.

1.3.1 Modélisation Physique : URDF et Xacro

Le robot n'existe virtuellement que si on lui donne un corps, une masse et des articulations. C'est le rôle du langage **URDF** (Unified Robot Description Format), souvent généré via **Xacro** (pour éviter la répétition de code).

Voici un extrait du code Xacro de notre robot décrivant son châssis principal (le corps) :

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="diff_robot">
    <link name="base_link">
        <visual>
            <origin xyz="0␣0␣0.075" rpy="0␣0␣0"/>
            <geometry>
                <box size="0.3␣0.3␣0.15"/> </geometry>
            <material name="white"/>
        </visual>
        <collision>
            <geometry>
                <box size="0.3␣0.3␣0.15"/>
            </geometry>
        </collision>
    </link>
</robot>
```

Listing 1.1 – Extrait de la description du châssis en URDF/Xacro[7]

1.3.2 Simulation 3D : Gazebo

Plutôt que de tester nos algorithmes sur un robot physique (avec les risques de casse associés), nous utilisons **Gazebo**. C'est un moteur physique puissant qui simule la gravité, le frottement des roues et les collisions.

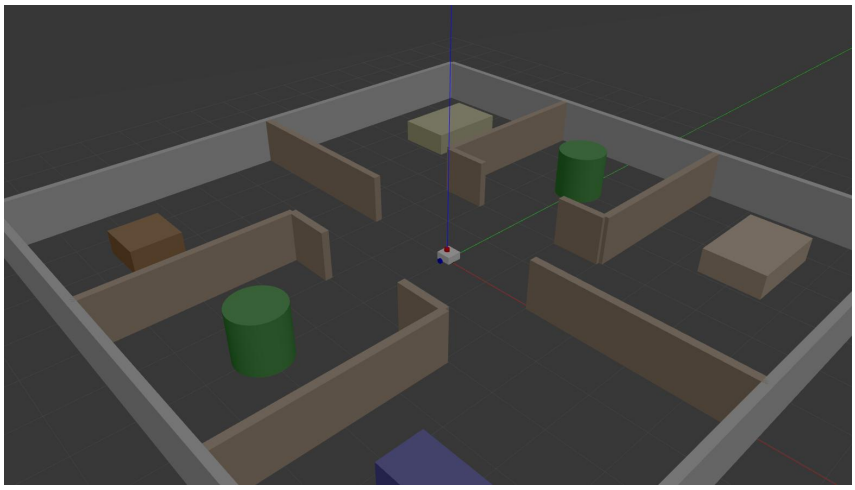


FIGURE 1.2 – Environnement de simulation du robot sous Gazebo 3D

1.3.3 Les Capteurs Virtuels : LiDAR 2D et IMU

Pour interagir avec Gazebo, le robot est équipé de deux capteurs fondamentaux :

— **Le LiDAR 2D :** Il émet un faisceau laser à 360 degrés pour calculer la distance des murs et des obstacles en temps réel.

Cette technologie génère des données en deux dimensions (axes X et Y) pour créer des nuages de points 2D ou des cartes de l'environnement, sans information de hauteur.

— **L'IMU (Centrale inertielle) :** Elle mesure l'accélération et l'orientation du robot, lui permettant de savoir s'il tourne ou s'il dérape.



FIGURE 1.3 – Scan laser par le LiDAR 2D

[8]



FIGURE 1.4 – Représentation des axes de l'IMU

[10]

1.3.4 Intelligence et Autonomie : Nav2 et SLAM

Une fois les données des capteurs récupérées, le "cerveau" du robot entre en jeu.

— Le **SLAM (Simultaneous Localization and Mapping)** construit la carte de la pièce tout en localisant le robot dedans.

— **Nav2 (Navigation 2)** utilise cette carte pour planifier un trajet sûr, générer des *Costmaps* (cartes de coût) et éviter les obstacles imprévus.

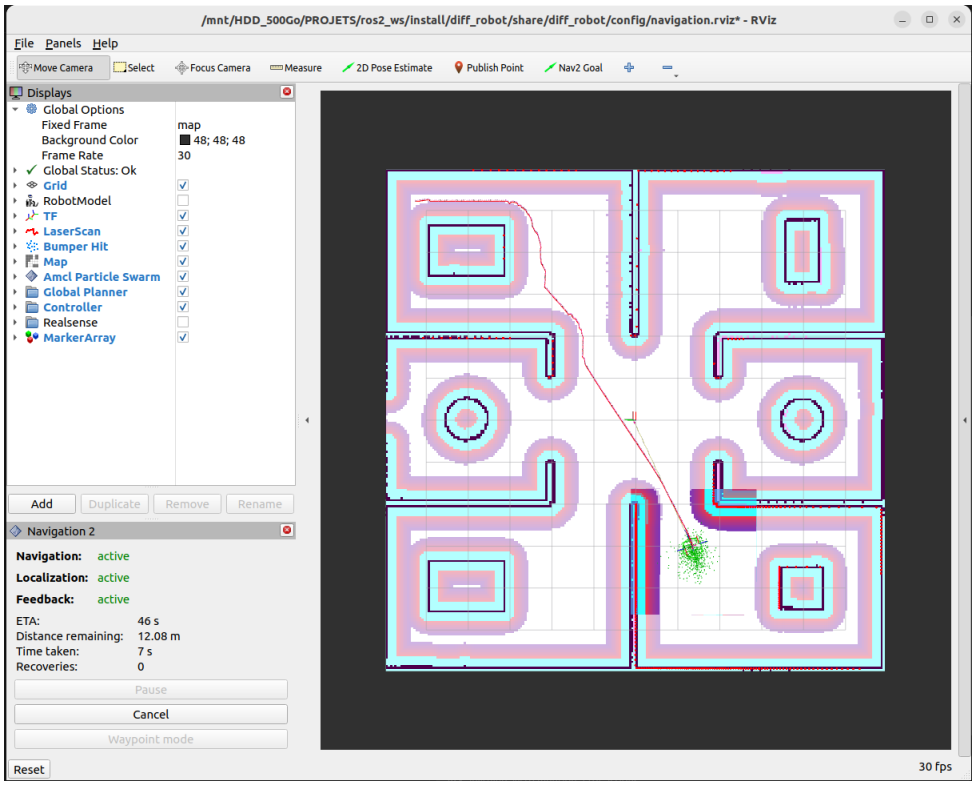


FIGURE 1.5 – Visualisation dans RViz2 de la carte générée (SLAM) et de la trajectoire (Nav2)

1.4 Le Workflow du Système

Pour illustrer le comportement de notre système autonome, la Figure 1.6 présente le flux de données continu entre la simulation, les capteurs et les algorithmes de décision.

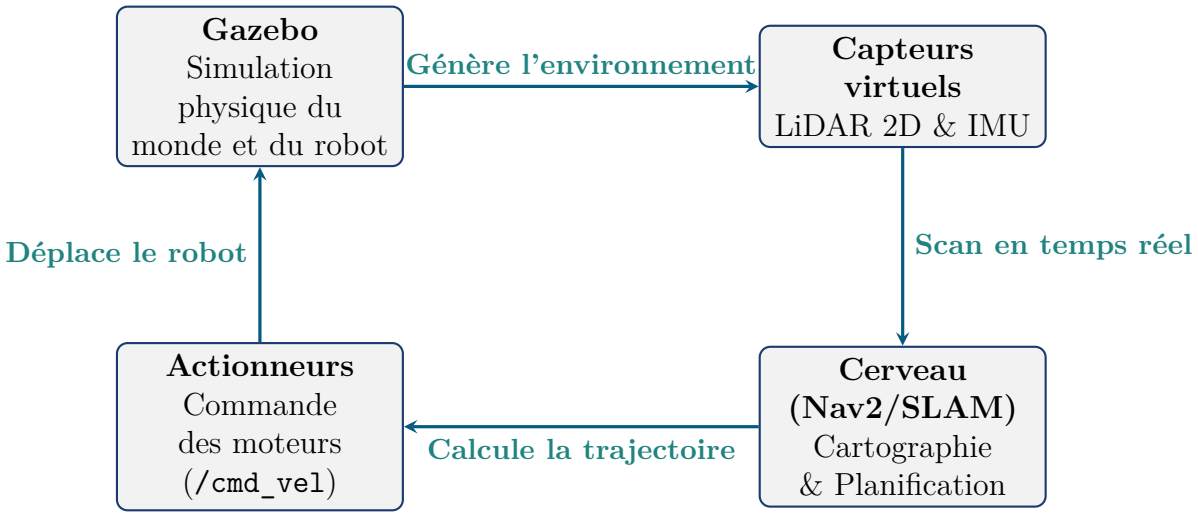


FIGURE 1.6 – Workflow de la boucle de rétroaction du robot autonome

1.5 Organisation de l'Équipe et Sprints

Pour mener à bien ce projet, le travail a été organisé selon une méthode agile en **Sprints**. Le tableau 1.2 détaille la répartition des rôles entre les pôles.

Pôle d'Expertise	Membres de l'équipe	Responsabilités clés
Navigation & Autonomie	Mickaella Raveloharisoa, Scidelo Andriasahy	Téléopération, Configuration Nav2, Gestion des Costmaps.
Perception & Modélisation	Kchit Hajar, Maimoun-Chekri, Laghouiyel	Modélisation URDF, Intégration Gazebo, SLAM visuel.

TABLE 1.2 – Répartition des rôles et organisation de l'équipe GSEA2

1.6 Structure du Rapport

Ce rapport est structuré de manière chronologique :

→ Le **Chapitre 2** détaillera l'environnement de travail collaboratif (Git/GitHub) et l'architecture logicielle.

→ Le **Chapitre 3** plongera dans le cœur de ROS 2 (Topics, Services, URDF).

→ Le **Chapitre 4** sera dédié au SLAM et aux capteurs.

→ Le **Chapitre 5** abordera la navigation autonome et l'évitement d'obstacles (Nav2).

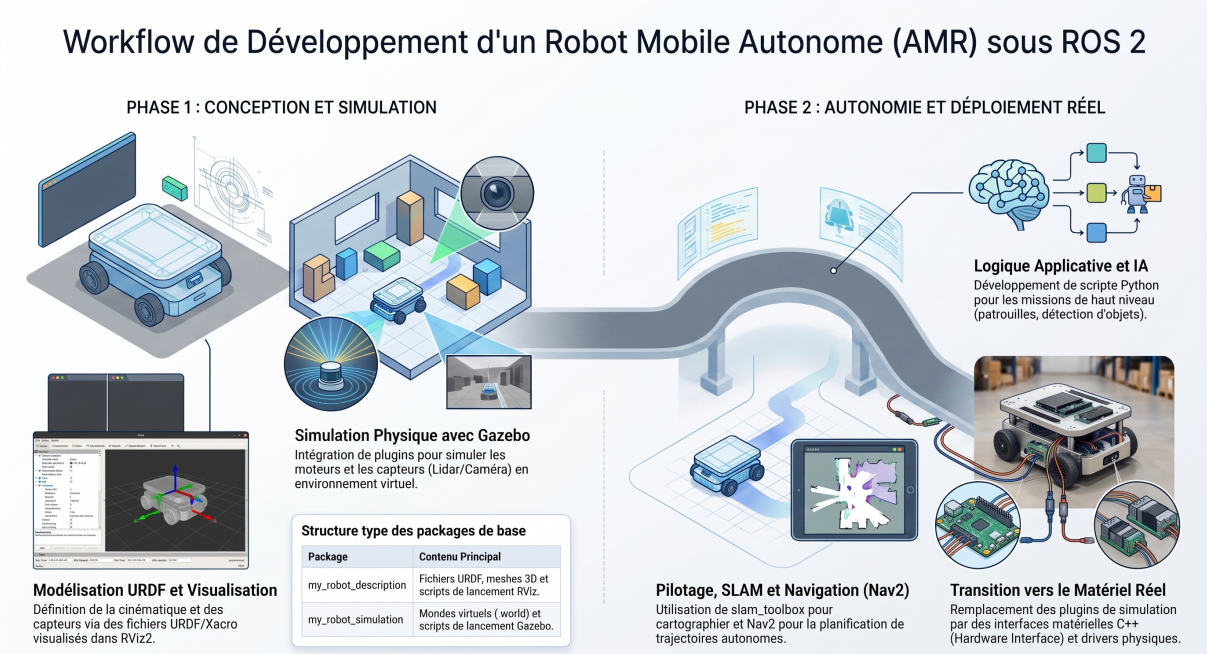


FIGURE 1.7 – Feuille de route globale (Roadmap) du développement du robot mobile sous ROS 2

Outil / Technologie	Catégorie	Définition et Rôle dans le projet
Ubuntu 22.04	Système d'Exploitation	L'environnement Linux de base, standard de l'industrie, optimisé et requis pour faire tourner notre version de ROS 2.
ROS 2 (Humble)	Middleware Robotique	Le cœur du projet. Il agit comme un système nerveux permettant à tous les composants (Nœuds) de communiquer en temps réel.
Gazebo	Moteur de Simulation 3D	Un puissant simulateur recréant la physique réelle (gravité, collisions, frottements) pour tester le robot sans risque de casse matérielle.
URDF & Xacro	Modélisation Physique	Formats de fichiers basés sur XML utilisés pour concevoir l'anatomie du robot (châssis, roues motrices, roue caster).
Nav2	Intelligence (Navigation)	La pile logicielle (stack) chargée de planifier les trajectoires globales et locales pour contourner les obstacles.
SLAM Toolbox	Algorithme (Cartographie)	L'outil permettant au robot de se repérer dans l'espace tout en dessinant la carte de son environnement en simultané.
RViz2	Visualisation de Données	Interface graphique permettant de voir "par les yeux du robot" (nuage de points LiDAR, carte générée, trajectoire prévue).
Python & C++	Langages de Scripting	Utilisés pour programmer la logique des Nœuds, notamment la capture des touches clavier pour la téléopération.
Git & GitHub	Travail Collaboratif	Outils de gestion de version indispensables pour centraliser, fusionner et protéger le code développé par les différents pôles de l'équipe.

TABLE 1.1 – Liste exhaustive des outils et technologies utilisés pour le robot mobile autonome

CHAPITRE 2

ENVIRONNEMENT DE TRAVAIL ET MÉTHODOLOGIE COLLABORATIVE

2.1 Infrastructure Système et Choix de Déploiement

La mise en place de l'infrastructure logicielle constitue la première étape critique de notre démarche d'ingénierie. Pour faire tourner la distribution **ROS 2 Humble Hawksbill**, le choix du système d'exploitation s'est naturellement porté sur **Ubuntu 22.04 LTS (Jammy Jellyfish)**.

En effet, chaque version de ROS 2 est intimement liée à une version spécifique du noyau Linux et des bibliothèques systèmes standards (`glibc`). Briser cet appairage expose le système à d'importantes erreurs de segmentation et d'incompatibilité de compilation.

2.1.1 Dilemme Technique : Machine Virtuelle vs Dual Boot

Au sein de l'équipe, deux philosophies de déploiement ont été explorées pour configurer nos stations de travail. Le choix entre une solution virtualisée et une installation native (Dual Boot) implique des arbitrages techniques majeurs en termes de performance, de stabilité matérielle et de sécurité.

DÉFINITION : Machine Virtuelle (VM)
Une VM est un ordinateur logiciel qui fonctionne à l'intérieur de votre ordinateur physique. Elle possède son propre système d'exploitation (Linux) isolé de l'hôte (Windows). Cependant, elle n'a pas d'accès direct à la carte graphique physique (GPU) de l'ordinateur.

Le tableau 2.1 dresse une analyse comparative rigoureuse de ces deux approches.

Critères d'analyse	Machine Virtuelle (VirtualBox)	Installation Native (Dual Boot)
Accès Matériel (GPU)	Virtualisé et limité. Pas d'accès direct au GPU physique.	Accès natif complet. Exploitation maximale des cartes NVIDIA/AMD.
Performances Gazebo	Faibles à moyennes. Ralentissements fréquents dus au rendu logiciel.	Optimales. Simulation fluide à haute fréquence (Hz).
Sécurité du Système	Absolue. Environnement cloisonné et sécurisé dans un fichier conteneur.	Risquée lors du partitionnement initial du disque (GRUB).
Flexibilité	Excellente. Sauvegardes instantanées (Snapshots) faciles.	Rigide. Nécessite un redémarrage complet de la machine hôte.

TABLE 2.1 – Comparatif technique entre le déploiement par Machine Virtuelle et Dual Boot

2.1.2 Gestion du Rendu Graphique sur VirtualBox

Pour les membres de l'équipe travaillant sur **VirtualBox (version 7.0.22)**, le manque d'accélération matérielle 3D native a provoqué des plantages systématiques (`code 255`) au lancement du moteur physique **Gazebo**. Pour contourner cette limitation, nous avons forcé le recours au rendu logiciel (exécution des calculs graphiques par le processeur principal CPU) en injectant une variable d'environnement :

```
# Forcer l'architecture a utiliser Mesa en mode logiciel
$ export LIBGL_ALWAYS_SOFTWARE=1
```

Listing 2.1 – Configuration du rendu logiciel graphique

2.2 Procédure d'Installation Étape par Étape

Afin d'assurer la reproductibilité de notre environnement de thèse, la configuration système suit scrupuleusement la méthode d'installation par paquets Debian officiels.

2.2.1 Configuration des Locales et des Dépôts Apt

ROS 2 nécessite un encodage système supportant nativement l'UTF-8. La première étape consiste à configurer les variables linguistiques (Locales) de Linux, puis à authentifier les serveurs de clés de l'Open Source Robotics Foundation (OSRF).

DÉFINITION : Clé GPG et Keyring
Une clé **GPG** agit comme un certificat de sécurité numérique. Elle garantit à votre gestionnaire de paquets (`apt`) que le code ROS 2 téléchargé provient bien des serveurs officiels et n'a pas été corrompu ou modifié par un tiers malveillant.

```
# Mettre a jour les index de paquets et forcer l'UTF-8
$ sudo apt update && sudo apt install locales -y
$ sudo locale-gen fr_FR fr_FR.UTF-8
$ sudo update-locale LC_ALL=fr_FR.UTF-8 LANG=fr_FR.UTF-8
$ export LANG=fr_FR.UTF-8

# Ajouter le depot officiel ROS 2 a Ubuntu
$ sudo apt install curl gnupg2 lsb-release -y
$ sudo curl -sSL https://raw.githubusercontent.com/ros2/rosdistro/master/
    ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg

# Injecter le depot dans les sources de l'Advanced Package Tool (APT)
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/
    keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(
    lsb_release -cs) main" | sudo tee /etc/apt/sources.list.data/ros2.list >
     /dev/null
```

Listing 2.2 – Initialisation du système et configuration des clés de sécurité

2.2.2 Installation de ROS 2 Humble et des Packages d'Intelligence

Une fois les dépôts synchronisés, nous installons la version **Desktop** qui embarque le cœur de ROS, les outils de communication, ainsi que les interfaces graphiques **RViz2** et **Gazebo**. Ensuite, nous installons les briques logicielles indispensables au travail de nos pôles : la stack de navigation autonome **Nav2** et la suite cartographique **SLAM Toolbox**.

```
# Actualiser la liste des paquets suite a l'ajout du depot
$ sudo apt update

# Installation de la base ROS 2 Desktop (Core + RViz2 + Gazebo)
$ sudo apt install ros-humble-desktop -y

# Installation des paquets dedies a la navigation et a la cartographie
$ sudo apt install ros-humble-navigation2 ros-humble-nav2-bringup ros-
    humble-slam-toolbox -y

# Installation des outils de compilation requis (Colcon)
$ sudo apt install python3-colcon-common-extensions -y
```

Listing 2.3 – Installation de la stack robotique globale

2.2.3 Persistance de l'Environnement (Sourcing)

Chaque nouveau terminal Linux ouvert ignore l'existence de ROS 2 tant qu'il n'a pas chargé ses scripts de configuration. Pour éviter de retaper manuellement la commande de liaison, nous automatisons le processus en modifiant le script de démarrage caché de l'utilisateur : le fichier `.bashrc`.

```
# Ajouter la configuration de base au profil de l'utilisateur
$ echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc

# Ajouter l'automatisation du rendu logiciel pour les utilisateurs sous VM
```

```
$ echo "export LIBGL_ALWAYS_SOFTWARE=1" >> ~/.bashrc

# Recharger instantanement le fichier de configuration sans fermer le
    terminal
$ source ~/.bashrc
```

Listing 2.4 – Automatisation du sourcing environnemental

2.3 L'Espace de Travail ROS 2 (Workspace)

Dans ROS 2, le code ne peut pas être exécuté depuis n'importe quel dossier. Il doit être structuré au sein d'un **Workspace** (espace de travail), que nous avons nommé `ros2_ws`.

Cet espace est divisé en plusieurs sous-dossiers stratégiques. Tout notre code source (fichiers URDF, scripts Python) est obligatoirement placé dans le répertoire `src/`. La compilation de l'ensemble des paquets est réalisée grâce à l'outil **Colcon** :

```
# Se placer à la racine de l'espace de travail
$ cd ~/ros2_ws
# Compiler en créant des liens symboliques (évite de recompiler pour chaque
    modif Python)
$ colcon build --symlink-install
# Sourcer l'environnement pour que le terminal reconnaisse les nouveaux
    noeuds
$ source install/setup.bash
```

Listing 2.5 – Compilation de l'espace de travail ROS 2

2.3.1 L'arborescence standard d'un espace de travail

La structure d'un espace de travail ROS 2, suit une arborescence standard et bien définie. Celle-ci permet d'organiser clairement les sources, les fichiers de compilation et les artefacts générés. La figure suivante présente l'organisation typique d'un workspace ROS 2 ainsi que la structure détaillée d'un package :

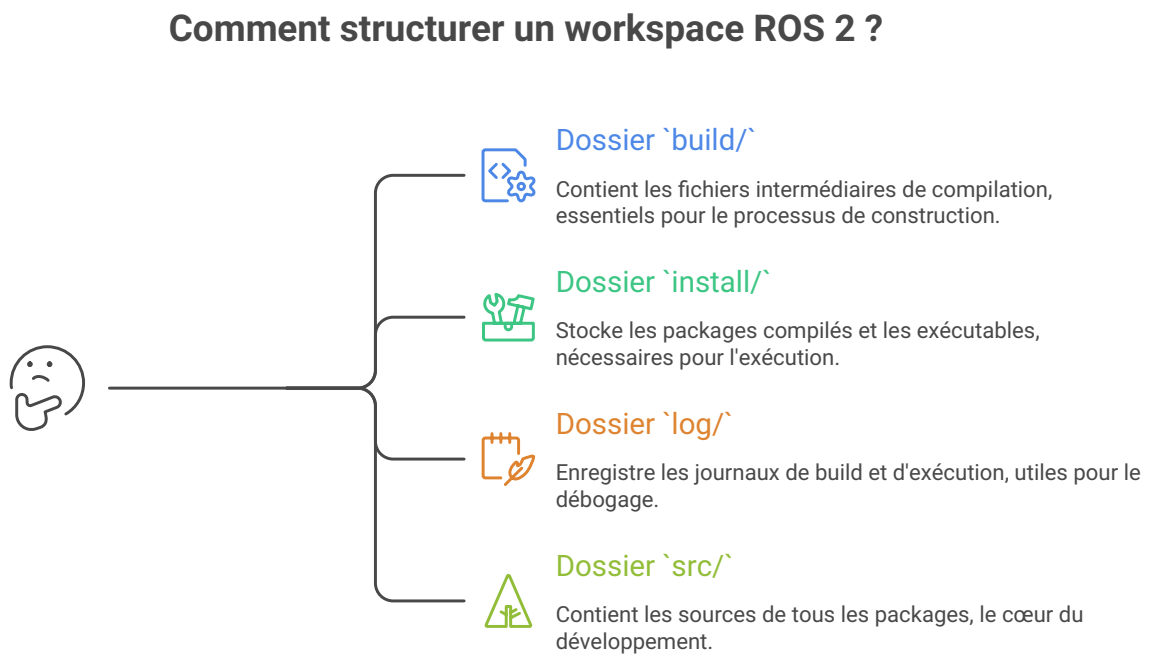


FIGURE 2.1 – Arborescence standard d'un workspace ROS 2

2.3.2 Création et Compilation d'un Package Python

Pour initialiser un nouveau module fonctionnel au sein du Workspace (comme notre paquet `diff_robot`), la commande standard de l'écosystème ROS 2 exige de spécifier le système de build souhaité. Dans notre cas, nous utilisons **ament_python** :

```
# Se placer dans le repertoire source avant de creer le paquet
$ cd ~/ros2_ws/src
$ ros2 pkg create <nom_du_package> --build-type ament_python
```

Listing 2.6 – Création d'un nouveau package ROS 2 en Python

Cette commande agit comme un générateur de patron (*scaffolding*). Elle génère automatiquement les fichiers de configuration administratifs essentiels : `package.xml`, `setup.py` et `setup.cfg`, ainsi que le répertoire principal `<nom_du_package>/` destiné à contenir le script du nœud (Node) principal.

Une fois le code source rédigé, la compilation s'effectue depuis la racine de l'espace de travail. Pour gagner un temps précieux lors du développement, il est courant de ne compiler que le paquet sur lequel l'équipe travaille actuellement grâce à l'argument `--packages-select` :

```
$ cd ~/ros2_ws
$ colcon build --packages-select <nom_du_package> --symlink-install
```

Listing 2.7 – Compilation ciblée d'un package spécifique avec Colcon

2.3.3 Analyse de l'Arborescence du Package Autonome

Afin de comprendre l'organisation interne de notre application, le listing 2.8 expose la structure complète de notre dépôt Git principal situé dans `autonomous-mobile-robot-`

ros2. Ce package suit scrupuleusement les spécifications du système de build **ament_python**.

```
autonomous-mobile-robot-ros2/
 .git/                  # Dossier de controle de version Git
 .gitignore             # Fichiers et dossiers a exclure du suivi Git
 ANALYSE_PROJET.md      # Notes de synthese et revues techniques internes
 ros2_video1.zip        # Archives de ressources et sauvegardes medias
 src/                   # Répertoire racine du code du package ROS 2
      config/             # Parametres de navigation (Nav2), RViz, SLAM et
   controleurs (.yaml)
      description/       # Modelisation géométrique et physique du robot (
   URDF/Xacro)
      diff_robot/        # Coeur logique : scripts Python des Noeuds ROS 2
   de l'équipe
      launch/            # Scripts Python d'orchestration (.launch.py)
          navigation.launch.py
          rsp_gazebo.launch.py
      maps/              # Cartes 2D de l'environnement (.yaml et .pgm)
   issues du SLAM
      resource/          # Fichiers index d'identification pour l'
   environnement ament
      test/              # Algorithmes de tests unitaires et de non-
   régression
      world/             # Definition des mondes 3D et des obstacles sous
   Gazebo
      package.xml        # Manifeste XML definissant les métadonnées et
   dependances apt
      setup.cfg          # Fichier de configuration des scripts de
   deploiement setuptools
      setup.py           # Script d'installation Python declarant les
   points d'entree
```

Listing 2.8 – Structure complète et hiérarchique du package robotique

Pour garantir une maintenance modulaire du robot, chaque répertoire de ce package assume un rôle technique isolé et bien défini :

- **src/description/ :** Ce dossier contient l'anatomie géométrique, visuelle et inertielle du robot. C'est ici que sont définis les fichiers URDF et Xacro décrivant les liaisons cinématiques entre le châssis principal, les roues motrices et la roue folle (caster).
- **src/launch/ :** Contient les scripts Python d'orchestration. Par exemple, le fichier `rsp_gazebo.launch.py` permet d'invoquer en une seule ligne de commande le nœud de gestion d'état (*Robot State Publisher*), de lancer le simulateur Gazebo 3D, et d'injecter (spawner) notre robot dans son monde physique.
- **src/world/ :** Héberge la configuration spatiale 3D de l'environnement dans lequel le robot est testé. Il décrit la position des murs, des labyrinthes, de l'éclairage et des obstacles dynamiques.
- **src/diff_robot/ :** Constitue le noyau applicatif de notre projet. Il s'agit du module Python qui renferme l'ensemble des scripts exécutables développés par l'équipe (notamment le script de téléopération clavier).
- **src/config/ :** Regroupe les hyperparamètres au format YAML. C'est ici que nous configurons finement les tolérances des contrôleurs de trajectoire, l'affichage per-

sonnalisé de RViz2, ainsi que la taille des *Costmaps* de Nav2 (comme le rayon de sécurité du robot) et en plus les parametres du SLAM.

- **`package.xml` & `setup.py` :** Ces fichiers matérialisent le squelette administratif de ROS 2. Le manifeste `package.xml` liste de façon stricte les dépendances logicielles requises (`rclpy`, `nav2`, etc.), tandis que `setup.py` enregistre les points d'entrée (alias) pour permettre au terminal d'exécuter nos scripts via la commande `ros2 run`.

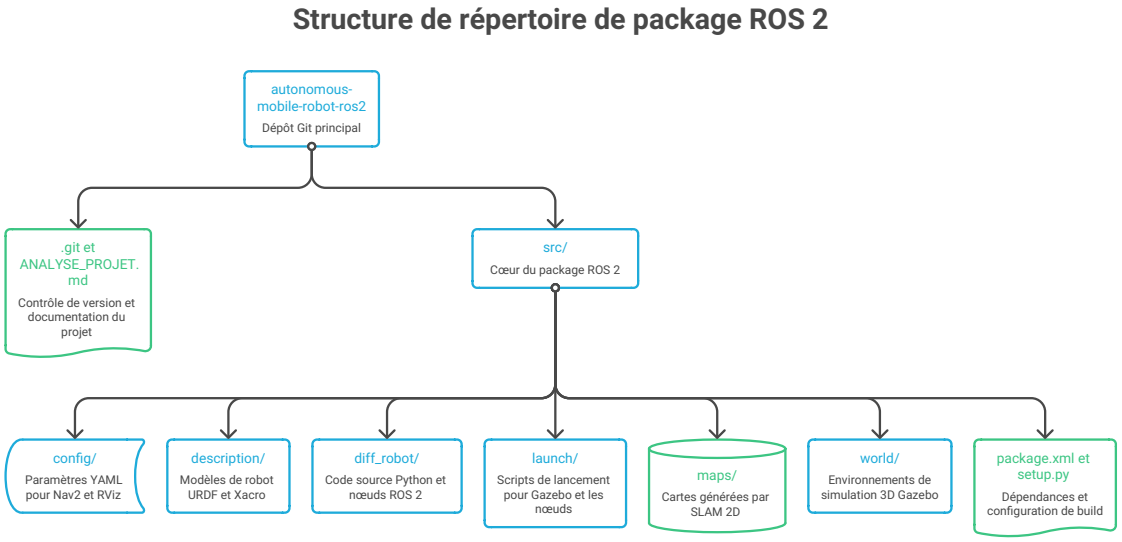


FIGURE 2.2 – Représentation visuelle de l'arborescence du package ROS 2 de notre projet

2.4 Gestion de Version et Synchronisation de l'Équipe

Travailler à cinq sur le même code source (Kchit, Maimoun-Chekri, Raveloharisoa, Laghouiyel et Andriasahy) nécessite une rigueur absolue pour éviter d'écraser le travail des autres. Nous avons donc mis en place un système de contrôle de version basé sur **Git** et hébergé sur **GitHub**.

2.4.1 Le Workflow Git/GitHub (Logigramme)

La Figure 2.3 illustre le cycle de développement continu adopté par notre équipe. Le dépôt GitHub central sert de vérité unique. Chaque membre clone ce dépôt localement, développe sa fonctionnalité (par exemple, la Costmap), teste en simulation, puis pousse (Push) ses modifications vers le serveur.

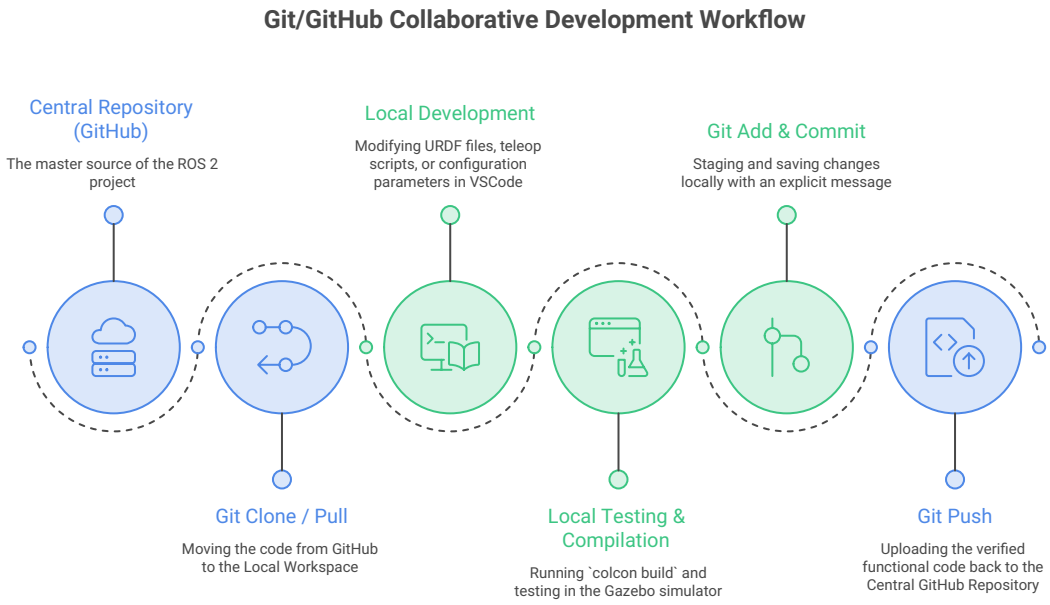


FIGURE 2.3 – Workflow de synchronisation collaborative avec Git et GitHub

2.4.2 Standardisation des Commandes (Terminal vs Éditeur)

Pour gagner en productivité, notre équipe utilise l'environnement de développement **Visual Studio Code (VSCode)**. L'interface graphique de VSCode intègre nativement les commandes Git, ce qui simplifie la gestion des versions.

Le tableau 2.2 synthétise la correspondance entre les commandes théoriques (Terminal) et nos actions pratiques au quotidien.

Action de l'équipe	Commande Terminal (Théorie)	Action dans VSCode (Pratique)
Télécharger le projet	`git clone [lien_https]`	`Ctrl+Shift+P` > *Git : Clone*
Récupérer les nouveautés	`git pull`	Menu ... > *Tirer (Pull)*
Préparer les fichiers	`git add .`	Cliquer sur le + des fichiers modifiés
Sauvegarder en local	`git commit -m "msg"`	Taper le message + Bouton *Valider*
Envoyer à l'équipe	`git push`	Bouton *Synchroniser les modifs*

TABLE 2.2 – Traduction des commandes de versionnage : Théorie vs Pratique

2.5 Approche Agile : Le Découpage en Sprints

La gestion de ce projet s'est inspirée de la **Méthodologie Agile**, en divisant la charge de travail en cycles de développement courts appelés **Sprints**.

- **Sprint 1 (Fondations et Téléopération) :** L'objectif était de mettre en place le dépôt GitHub, de modéliser le robot via URDF et de développer le nœud `robot_teleop` permettant de contrôler le robot manuellement au clavier dans Gazebo.
- **Sprint 2 (Perception et Navigation) :** Le pôle Perception s'est concentré sur l'implémentation du *SLAM Toolbox* pour générer la carte. En parallèle, le pôle Navigation a configuré les paramètres critiques de *Nav2* (comme le `robot_radius` et l'`inflation_radius`) pour préparer l'évitement d'obstacles autonome.

Cette approche modulaire nous a permis de valider étape par étape le comportement du robot avant de l'enrichir de nouvelles fonctionnalités d'intelligence artificielle.

CHAPITRE 3

FONDEMENTS LOGIQUES ET MODÉLISATION PHYSIQUE

3.1 Le Graphe de Calcul : L'écosystème ROS 2

Contrairement à un programme informatique classique où un seul grand script exécute toutes les tâches, l'architecture de **ROS 2** repose sur une philosophie de **décentralisation absolue**. Le système global (le robot) est divisé en dizaines de petits programmes indépendants appelés **Nœuds** (*Nodes*).

L'ensemble de ces Nœuds connectés forme ce que l'on appelle le **Graphe de Calcul** (*Computational Graph*).

DÉFINITION : Un Nœud (Node) ROS 2
Un Nœud est un processus exécutable qui a une tâche unique et bien précise. Par exemple, un robot aura un nœud `camera_node` (qui lit la vidéo), un nœud `teleop_node` (qui écoute le clavier), et un nœud `motor_node` (qui fait tourner les roues). Si le nœud de la caméra plante, le nœud des moteurs continue de fonctionner !

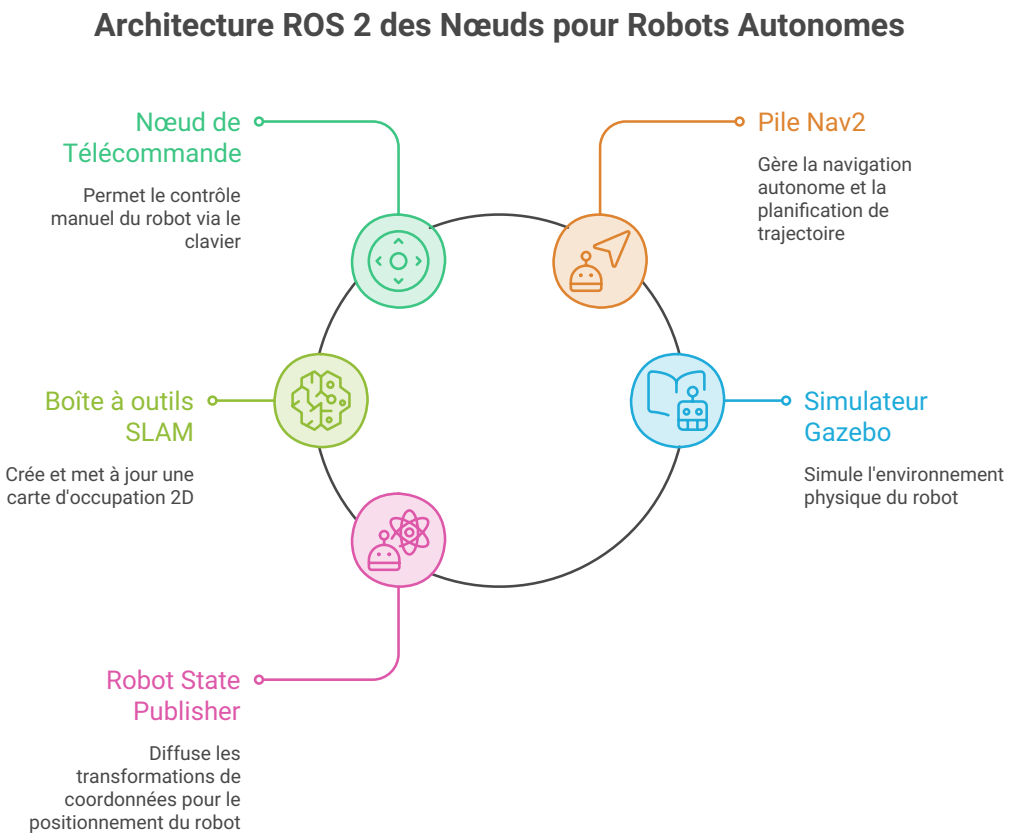


FIGURE 3.1 – Architecture logicielle globale : Nœuds ROS 2 et échanges via les Topics

3.2 Communication et Échange de Données

Pour que le robot fonctionne intelligemment, ces nœuds isolés doivent pouvoir s'échanger des informations (messages) de manière fluide. Le middleware **DDS** de ROS 2 propose deux mécanismes principaux pour cela : les **Topics** et les **Services**.

3.2.1 Les Topics : Flux de données continu

Les Topics utilisent un modèle **Publisher / Subscriber** (Éditeur / Abonné). C'est un système de diffusion unidirectionnelle (comme une chaîne de télévision) :

- Le **Publisher** envoie des données en continu sur un "canal" (le Topic), sans savoir si quelqu'un l'écoute.
- Le **Subscriber** s'abonne à ce canal pour lire les données.

Exemple du projet : Le capteur LiDAR publie des milliers de points par seconde sur le topic `/scan`. L'algorithme SLAM s'y abonne pour dessiner la carte.

3.2.2 Les Services : Requête et Réponse

Contrairement aux Topics, les Services utilisent un modèle **Client / Serveur**. C'est une communication bidirectionnelle et synchrone (comme un appel téléphonique) :

- Le **Client** envoie une requête spécifique et **attend** que la tâche soit accomplie.

- Le **Serveur** reçoit la requête, exécute l'action, et renvoie une réponse (Succès ou Échec).

Exemple du projet : Demander à Gazebo de faire apparaître (spawner) le robot au lancement avec le service `/spawn_entity`.

Le tableau 3.1 et les figures (Figure 3.2 et Figure 3.3) résument ces deux paradigmes vitaux.

Caractéristique	Topics (Pub/Sub)	Services (Client/Serveur)
Type de Flux	Continu (Stream), Asynchrone.	Ponctuel (One-shot), Synchrone.
Relation	1 vers N (Un émetteur, plusieurs récepteurs).	1 vers 1 (Un client, un serveur).
Cas d'usage idéal	Données de capteurs (LiDAR, Caméra, Odométrie).	Actions critiques (Allumer un laser, sauvegarder une carte).

TABLE 3.1 – Comparaison technique entre les Topics et les Services ROS 2

3.2.3 Illustration des Communications

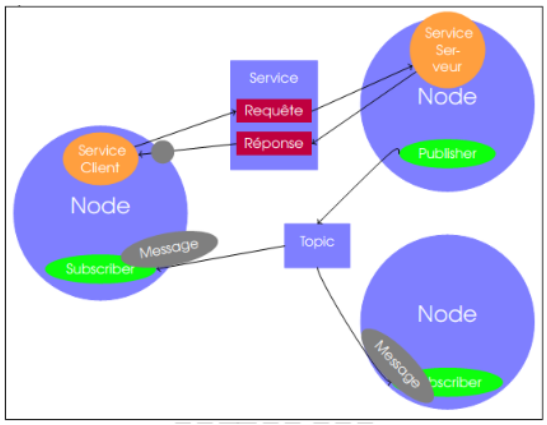


FIGURE 3.2 – Modèle Publisher / Subscriber (Topic)[5]

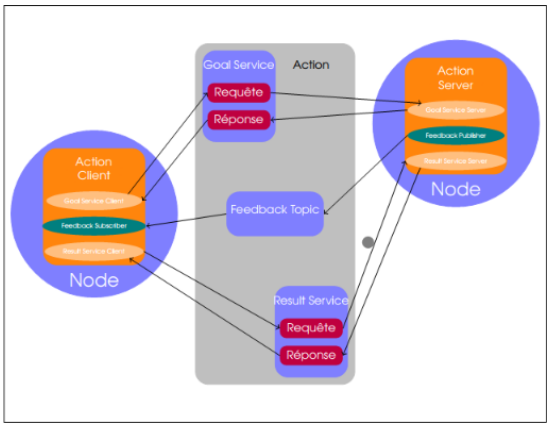


FIGURE 3.3 – Modèle Client / Serveur (Service)[5]

3.3 Modélisation Physique : De l'URDF à Xacro

Avant de pouvoir naviguer, le robot doit exister physiquement dans la simulation. Pour cela, ROS 2 utilise un format de fichier standard basé sur XML appelé **URDF** (Unified Robot Description Format).

Cependant, écrire un robot complexe en URDF pur est laborieux et source d'erreurs (répétition de code, valeurs écrites en dur). Pour pallier ce problème, notre équipe a modélisé le robot `diff_robot` en utilisant **Xacro** (XML Macros).

3.3.1 La puissance de Xacro

Xacro est un compilateur de macros qui permet de transformer l'URDF en un véritable langage de programmation. Il nous a permis d'utiliser :

→ **Des Propriétés (Variables) :** Par exemple, définir `<xacro:property name="wheel_radius" value="0.033"/>`. Si on change la taille de la roue, la modification s'applique partout instantanément.

→ **Des Fonctions Mathématiques :** Xacro permet de calculer automatiquement des inerties complexes (via `${pi}`, `${mass}`, etc.).

→ **Des Macros (Fonctions) :** Au lieu de coder la roue droite puis la roue gauche, nous avons créé une seule macro `wheel` que nous appelons deux fois, comme illustré dans l'extrait ci-dessous.

```
<xacro:macro name="wheel" params="prefix side">
    <link name="${prefix}_wheel_link">
        <visual>
            <geometry>
                <cylinder radius="${wheel_radius}" length="${wheel_length}" />
            </geometry>
            <material name="black"/>
        </visual>
        <collision>
            <geometry>
                <cylinder radius="${wheel_radius}" length="${wheel_length}" />
            </geometry>
        </collision>
    </link>

    <joint name="${prefix}_wheel_joint" type="continuous">
        <parent link="base_link"/>
        <child link="${prefix}_wheel_link"/>
        <axis xyz="0 1 0"/>
    </joint>
</xacro:macro>

<xacro:wheel prefix="left" side="1"/>
<xacro:wheel prefix="right" side="-1"/>
```

Listing 3.1 – Extrait Xacro : Macro pour la génération dynamique des roues

3.3.2 Architecture Modulaire du Modèle Xacro

Plutôt que de concevoir le robot dans un seul fichier monolithique géant (difficile à maintenir et à déboguer), le pôle Infrastructure a adopté une approche d'ingénierie logicielle modulaire. Le modèle physique est éclaté en plusieurs sous-fichiers spécialisés, tous appelés par un fichier maître `core_robot.xacro.urdf` :

- `inertial.xacro` : Contient les macros mathématiques définissant les matrices d'inertie (cylindres, boîtes, sphères) indispensables pour que le moteur physique de Gazebo calcule correctement les forces et les collisions.
- `robot_include.xacro` : Définit le corps principal du robot (le châssis, les roues motrices et la roue folle).

- `lidar.xacro & imu.xacro` : Instancient les liens (*links*) et articulations (*joints*) des capteurs virtuels sur le sommet du robot, tout en incluant leurs plugins Gazebo respectifs pour simuler les faisceaux lasers et les mesures d'accélération.
- `control.xacro` : Intègre le plugin `libgazebo_ros_diff_drive.so`. C'est ce fichier qui fait le pont entre le contrôleur ROS 2 et les roues physiques dans Gazebo, en calculant l'odométrie à partir de la rotation théorique des moteurs.

Cette séparation stricte garantit un code propre, réutilisable, et permet d'ajouter ou de retirer un capteur (comme une caméra) en ajoutant simplement une ligne `<xacro:include>` dans le fichier maître.

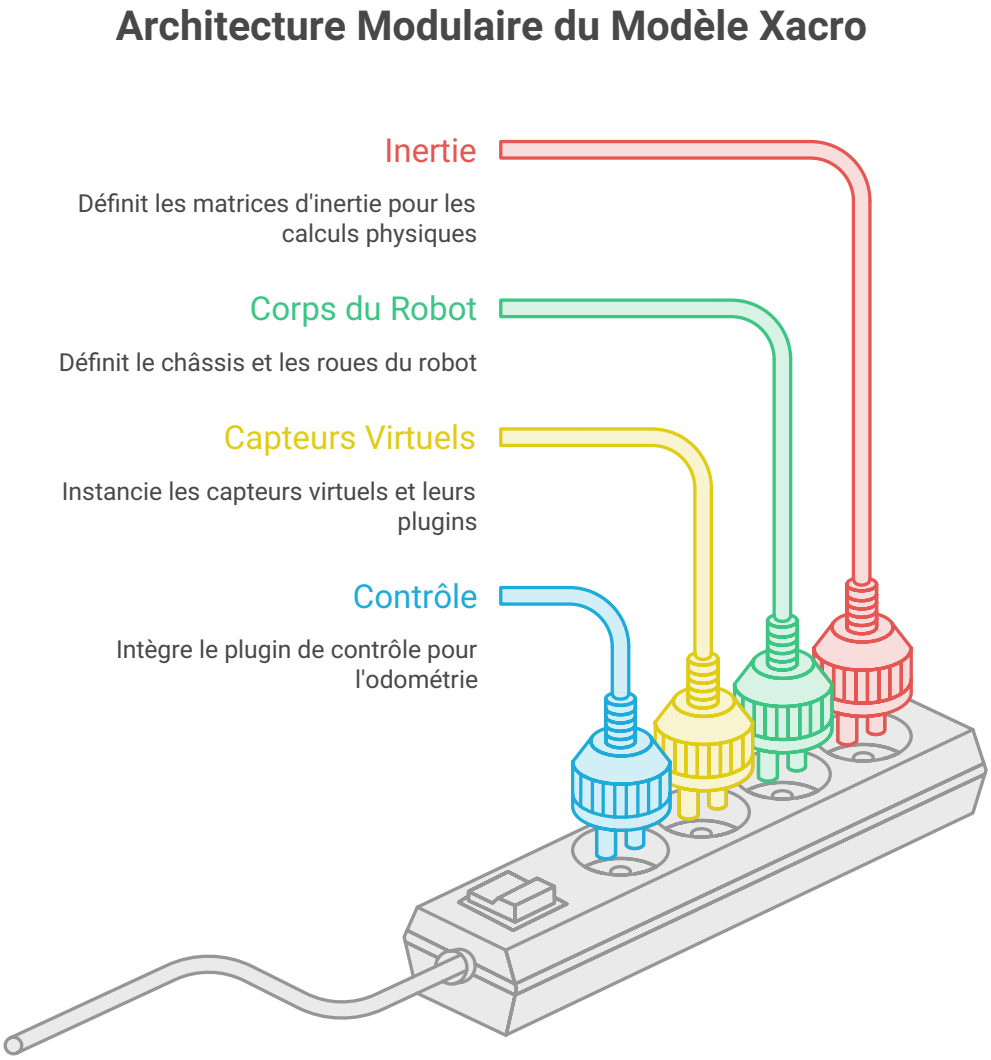


FIGURE 3.4 – Architecture Modulaire du Modèle Xacro

3.3.3 Nomenclature des Composants Physiques

L'anatomie de notre robot virtuel est structurée autour de plusieurs corps rigides (*Links*) reliés par des articulations (*Joints*). Le tableau 3.2 dresse l'inventaire des éléments physiques modélisés en URDF.

Composant	Fichier Xacro	Rôle et Caractéristiques
Châssis (Base)	`robot_include.xacro`	Structure principale (boîte $30 \times 30 \times 15\,cm$) portant le reste des composants.
Roues Motrices	`robot_include.xacro`	Deux roues cylindriques indépendantes gérant la propulsion et la rotation.
Roue Caster	`robot_include.xacro`	Roue sphérique folle à l'avant assurant l'équilibre avec un coefficient de frottement nul.
Capteur LiDAR	`lidar.xacro`	Cylindre monté sur le châssis, agissant comme le capteur de distance principal.
Capteur IMU	`imu.xacro`	Puce virtuelle au centre du robot captant l'accélération et l'orientation 3D.

TABLE 3.2 – Inventaire architectural des composants physiques du robot

3.3.4 L'Interface Physique-Logique : Les Plugins Gazebo

Pour que les fichiers URDF inertes prennent vie dans le simulateur 3D, le framework s'appuie sur des **Plugins Gazebo**. Ces bibliothèques dynamiques (`.so`) en C++ interceptent les données de Gazebo et les convertissent en messages ROS 2 :

- `libgazebo_ros_diff_drive.so` : S'abonne aux commandes de vitesse (`/cmd_vel`), applique des forces de rotation aux articulations des roues motrices, et calcule l'odométrie estimée (`/odom`).
- `libgazebo_ros_ray_sensor.so` : Simule le rebond des faisceaux laser sur les obstacles 3D et publie les distances sous forme de nuage de points sur le topic `/scan`.
- `libgazebo_ros_imu_sensor.so` : Calcule la gravité et les forces d'inertie subies par le modèle pour publier l'état sur `/imu`.

3.4 Cinématique Différentielle (Differential Drive)

L'architecture mécanique choisie pour notre projet est celle d'un **robot à entraînement différentiel**. Cela signifie que le robot ne possède pas de volant directionnel (comme une voiture).

Il se déplace grâce à deux roues motrices indépendantes (gauche et droite), soutenues par une roue folle (caster wheel) à l'avant pour l'équilibre.

- Pour aller tout droit : les deux roues tournent à la même vitesse dans le même sens.
- Pour tourner sur place : une roue tourne en avant, l'autre en arrière.

Afin que le simulateur **Gazebo** comprenne cette physique, nous avons intégré le plugin ROS `libgazebo_ros_diff_drive.so` directement dans nos fichiers Xacro. Ce plugin agit

comme le pont vital entre les commandes de vitesses virtuelles (Topic `/cmd_vel`) et la rotation effective des roues dans le simulateur 3D.

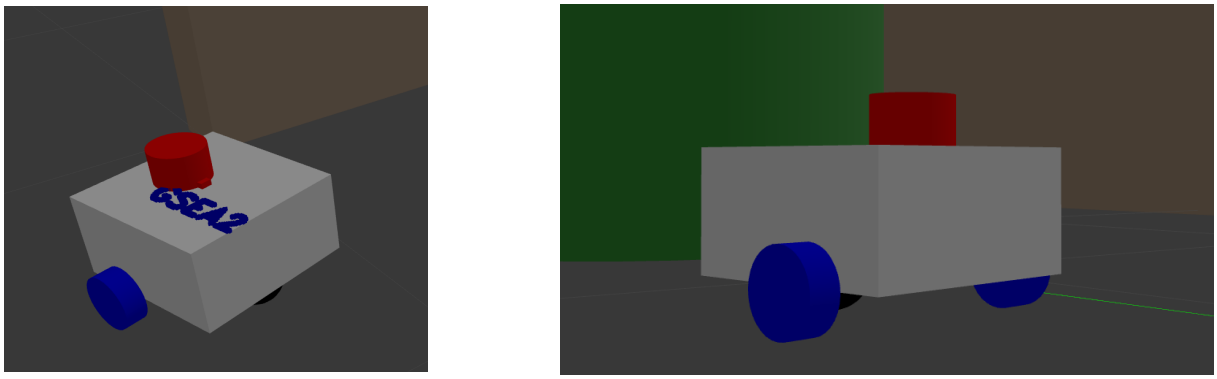


FIGURE 3.5 – Robot à entraînement différentiel dans Gazebo

CHAPITRE 4

PILOTAGE, PERCEPTION ET CARTOGRAPHIE SIMULTANÉE (SLAM)

4.1 Le Système de Téléopération Clavier (`robot_teleop`)

L'implémentation du nœud de téléopération manuelle `robot_teleop` constitue le livrable majeur du **Sprint 1**. Avant de confier l'autonomie totale au robot, il est indispensable de disposer d'un moyen de contrôle direct pour valider la physique du châssis dans **Gazebo**, tester l'adhérence des roues motrices et vérifier le comportement de la roue folle (*caster wheel*).

4.1.1 L'Architecture d'Exécution et Isolation des Processus

Dans l'écosystème ROS 2, il est impératif d'isoler les processus d'exécution au sein de terminaux Linux distincts. Le premier terminal est dédié à l'orchestration globale de la simulation via le script `rsp_gazebo.launch.py`. Le second terminal exécute de manière exclusive le nœud de téléopération :

```
$ cd ~/ros2_ws
$ source install/setup.bash
$ ros2 run diff_robot teleop
```

Listing 4.1 – Lancement du nœud de téléopération clavier

Cette séparation est indispensable sur le plan logiciel : le nœud de téléopération doit prendre le contrôle exclusif de l'interface d'entrée standard (`stdin`) du terminal afin d'intercepter instantanément chaque pression de touche clavier, sans parasitage par les flux de logs continus générés par le moteur physique de Gazebo.

4.1.2 Analyse Algorithmique du Script `teleop.py`

Le script repose sur une boucle d'échantillonnage asynchrone qui capture les entrées utilisateur et les traduit en consignes cinématiques normalisées.

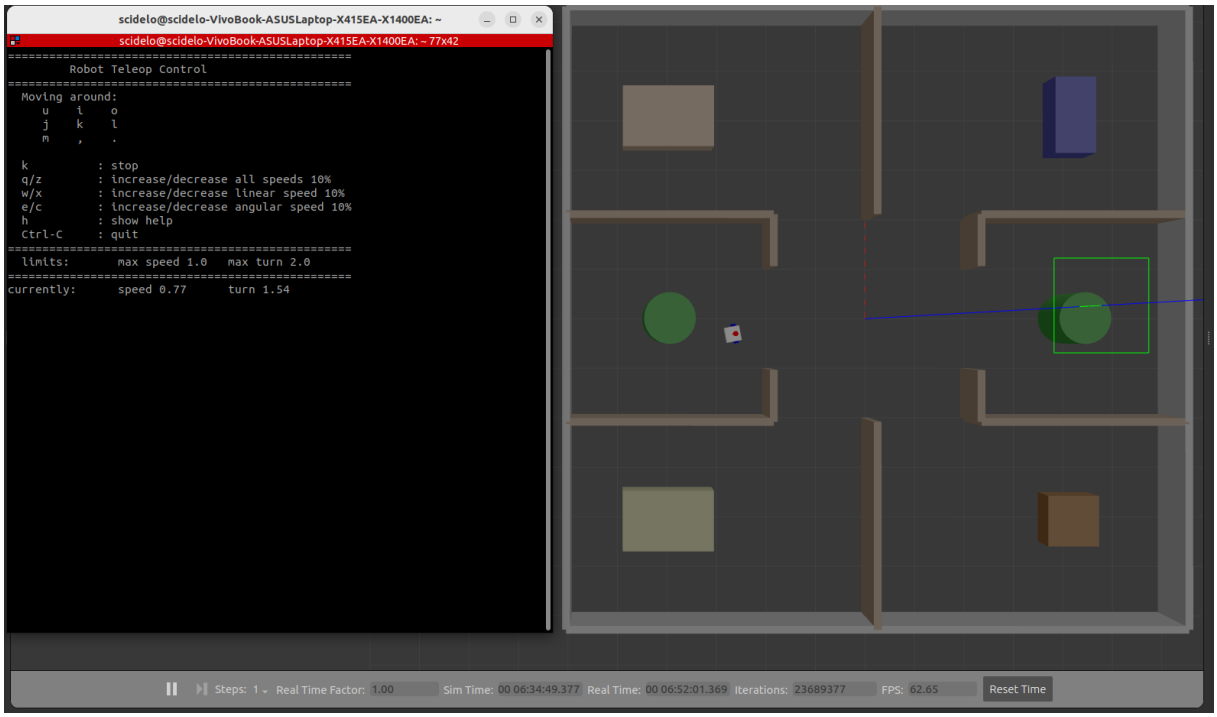


FIGURE 4.1 – Téléopération Clavier

1. Capture Non-Bloquante : `get_key()`

Pour éviter que le script ne se fige en attendant une entrée utilisateur, la fonction de capture modifie temporairement les attributs du terminal Linux via le module `termios`. Elle bascule le terminal en mode brut (*raw mode*), désactivant l'écho écran et la mise en mémoire tampon des lignes. Cela permet de lire chaque caractère au vol, de manière asynchrone.

2. Génération du Message Vectoriel : `geometry_msgs/msg/Twist`

Les commandes clavier (les touches `u, i, o, j, k, l, m, ,, .` pour les directions et `q, z` pour l'incrémentation des vitesses) sont associées à des matrices de gain. Ces gains viennent peupler un message de type **Twist**, qui est la structure standard de ROS pour transmettre des vitesses dans l'espace tridimensionnel :

- `linear.x` : Vitesse de translation longitudinale (en mètres par seconde, m/s).
- `angular.z` : Vitesse de rotation autour de l'axe vertical du robot (en radians par seconde, rad/s).

3. Sécurité Logique et Bridage (`Clamping`)

Pour garantir la stabilité de la simulation et la robustesse du nœud, deux mécanismes de sécurité critiques ont été programmés :

→ **Le Clamping (Bridage) :** Le script encadre les valeurs de vitesse afin d'éviter qu'une trop longue pression de touche n'engendre des accélérations infinies capables de faire chavirer le modèle URDF dans Gazebo.

→ **L'Interception des Anomalies :** Si l'utilisateur presse une touche non répertoriée dans les dictionnaires de correspondance (`moveBindings` ou `speedBindings`),

l'algorithme intercepte l'exception et publie immédiatement une consigne d'arrêt absolu : `Twist(0,0)`.

Comme le montre le listing 4.2, la structure du point d'entrée utilise un bloc défensif `try...finally`. Ce dernier garantit que même si l'utilisateur coupe le script brutalement via un signal d'interruption (`Ctrl+C`), la fonction `rclpy.shutdown()` est appelée pour nettoyer le graphe ROS 2 et couper l'alimentation virtuelle des actionneurs, évitant ainsi le phénomène de "nœud fantôme" maintenant le robot en mouvement perpétuel.

```
def main():
    try:
        # Initialisation du contexte ROS 2 et instanciation du noeud
        rclpy.init()
        teleop = RobotTeleop()
        teleop.run() # Boucle d'ecoute active du clavier
    except Exception as e:
        print(f"Erreur interceptee : {e}")
    finally:
        # Securite absolue : extinction propre du noeud et du middleware
        rclpy.shutdown()
```

Listing 4.2 – Structure sécurisée du point d'entrée de la téléopération

4.2 Perception Sensorielle : Les Capteurs Virtuels Embarqués

Au cours de son exploration manuelle, le robot récolte des données brutes en provenance de deux capteurs simulés de manière synchrone par Gazebo : le **LiDAR 2D** et la centrale inertielle **IMU**.

4.2.1 Le LiDAR 2D (Laser Scan)

Le LiDAR (*Light Detection and Ranging*) simulé projette un plan de faisceaux laser à 360 degrés autour de l'axe d'origine du robot. Il publie ses informations sur le topic `/scan` à une fréquence de 10 Hz, encapsulées dans des structures de type `sensor_msgs/msg/LaserScan`.

Ce message contient un tableau de distances (`ranges`). Chaque case représente la distance exacte (en mètres) séparant le centre du robot du premier obstacle rencontré (les cubes du labyrinthe virtuel) pour un angle donné. Ces données géométriques forment un nuage de points indispensable à la détection des frontières de l'environnement.

4.2.2 La Centrale Inertielle (IMU)

L'IMU (*Inertial Measurement Unit*) agit comme le système vestibulaire du robot. Placé au centre géométrique du châssis, ce capteur publie ses mesures sur le topic `/imu` via des messages `sensor_msgs/msg/Imu`. Il quantifie trois grandeurs physiques essentielles :

1. **L'accélération linéaire** (axes X, Y, Z) : pour mesurer les forces d'inertie.
2. **La vitesse angulaire** (axes X, Y, Z) : pour estimer le taux de rotation instantané.

3. **L'orientation spatiale** : exprimée sous forme de **Quaternions** (x, y, z, w) pour éviter les singularités mathématiques du blocage de cardan (*Gimbal Lock*) propres aux angles d'Euler.

L'intégration des données combinées de l'IMU et des codeurs de roues permet de générer une **Odométrie** (`/odom`) précise, indispensable pour que le robot estime son propre déplacement.

4.3 Cartographie SLAM : Implémentation et Algorithmique

Le défi majeur du pôle Perception (Sprint 3) a consisté à implémenter un système de Cartographie et Localisation Simultanées (SLAM) robuste. L'outil sélectionné pour cette tâche est **SLAM Toolbox**, une solution de pointe reposant sur l'algorithme **Karto SLAM**, qui utilise une approche d'optimisation par graphe (*Graph-based SLAM*).

4.3.1 Configuration Initiale et Fichiers Paramétriques

Afin de garantir la flexibilité du système sans altérer les fichiers natifs de ROS 2, le fichier de configuration par défaut a été déporté dans le package de notre projet.

```
# 1. Creation du repertoire de configuration s'il est inexistant
$ mkdir -p ~/ros2_ws/src/config

# 2. Copie du fichier des parametres asynchrones vers le projet
$ cp /opt/ros/humble/share/slam_toolbox/config/mapper_params_online_async.
    yaml ~/ros2_ws/src/config/
```

Listing 4.3 – Création du répertoire et copie du fichier de configuration SLAM

L'équipe Perception a ensuite procédé à un ajustement critique des variables environnementales au sein de ce fichier YAML pour l'adapter au robot :

- `odom_frame: odom` : Définit le repère de l'odométrie brute.
- `base_frame: base_footprint` : Repère d'origine au sol du robot, défini dans l'URDF.
- `scan_topic: /scan` : Cible le flux réel publié par le capteur LiDAR simulé.
- `use_sim_time: true` : Paramètre obligatoire sous Gazebo pour synchroniser les *timestamps* entre SLAM Toolbox et l'horloge virtuelle.

4.3.2 Fonctionnement Interne : Le Graphe d'Optimisation

Au-delà de son intégration, il est fondamental de comprendre le fonctionnement interne du processus de cartographie. Le pipeline de traitement des données suit une boucle logique de correction d'erreurs (voir Figure 4.2) :

1. **Association Temporelle (TF) :** À la réception d'un scan laser, l'algorithme l'associe instantanément à la pose du robot via la transformation `odom` → `base_footprint` au timestamp exact. C'est une étape critique : sans cette synchronisation, l'algorithme crashe en renvoyant l'erreur `Failed to compute odom pose`.

2. **Scan Matching :** Le scan actuel est mathématiquement comparé aux scans précédents de la carte pour estimer le déplacement réel et annuler la dérive inhérente à l'odométrie mécanique.
3. **Graphe Géométrique :** Chaque pose validée devient un *Nœud*, et le déplacement entre deux poses devient une *Arête* (contrainte) de ce graphe.
4. **Solveur Mathématique :** Périodiquement, lors d'une fermeture de boucle (*Loop Closure*), un plugin d'optimisation externe appelé **Ceres Solver** ajuste l'ensemble du graphe pour minimiser l'erreur cumulée.

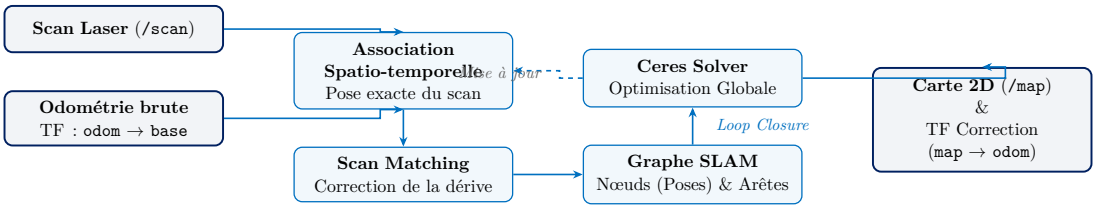


FIGURE 4.2 – Processus algorithmique interne : du capteur brut à la grille d'occupation (Karto SLAM)

4.4 Fondements Mathématiques de la Cartographie SLAM

La cartographie d'un environnement inconnu repose sur l'algorithme SLAM (Simultaneous Localization And Mapping). Dans ce projet, le package `slam_toolbox` s'appuie sur l'architecture logicielle de **Karto SLAM**, qui utilise une approche basée sur les graphes (Graph-SLAM) pour modéliser le problème.

4.4.1 Principe du Graph-SLAM et Scan Matching

L'approche Graph-SLAM divise le problème de cartographie en deux sous-systèmes : le *Front-end* et le *Back-end*.

Le *Front-end* est chargé de lier chaque scan laser reçu (`/scan`) à une pose estimée du robot via la transformation odométrique brute (`odom` → `base_footprint`) au moment exact du timestamp du capteur. L'algorithme effectue un *Scan Matching* : il compare le nouveau nuage de points aux scans précédents pour estimer le déplacement réel et corriger la dérive odométrique.

Chaque pose validée devient un **nœud** (x_i) dans le graphe, et chaque contrainte de déplacement (mesurée par l'odométrie ou la correspondance laser) devient une **arête** (z_{ij}).

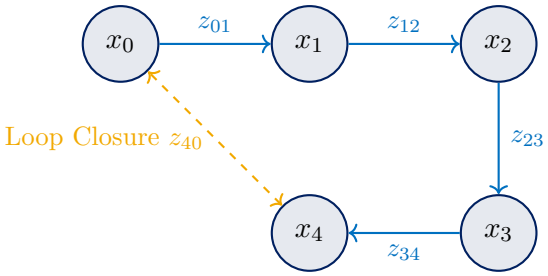


FIGURE 4.3 – Représentation topologique d'un Graph-SLAM avec fermeture de boucle

4.4.2 Optimisation du Graphe via Ceres Solver

Au fur et à mesure de l'exploration, l'erreur de pose s'accumule. Lorsque le robot repasse par une zone déjà visitée, l'algorithme détecte une "fermeture de boucle" (*Loop Closure*). C'est ici qu'intervient le *Back-end* mathématique.

L'objectif est de trouver la configuration optimale des poses $\mathbf{x}^*$ qui minimise la somme des erreurs résiduelles e_{ij} entre la mesure observée z_{ij} et la mesure prédite par le graphe $h(x_i, x_j)$. La fonction de coût non linéaire se formule ainsi :

$$\mathbf{x}^* = \arg\min_{\mathbf{x}} \sum_{\langle i,j \rangle \in \mathcal{C}} \Omega_{ij} \left\| z_{ij} - h(x_i, x_j) \right\|^2 \tag{4.1}$$

Où $\mathcal{C}$ représente l'ensemble des contraintes et Ω_{ij} la matrice d'information (inverse de la covariance de la mesure).

Pour résoudre ce système complexe, notre configuration de SLAM Toolbox repose exclusivement sur le solveur **Ceres Solver** (développé par Google). Les hyperparamètres définis dans notre fichier YAML garantissent une convergence rapide :

- **Solveur Linéaire (`SPARSE_NORMAL_CHOLESKY`)** : Adapté à la matrice jacobienne creuse (sparse) caractéristique des grands graphes de poses.
- **Préconditionneur (`SCHUR_JACOBI`)** : Accélère drastiquement la convergence du solveur sur les problèmes de type *Pose Graph Optimization*.
- **Stratégie d'optimisation (`LEVENBERG_MARQUARDT`)** : Algorithme itératif robuste alternant entre la méthode de Gauss-Newton et la descente de gradient pour ajuster les poses.

4.5 Génération Pratique de la Carte (Téléopération)

Pour que les mathématiques du SLAM s'exécutent efficacement, le robot doit acquérir des données environnementales de haute qualité. La méthode de balayage spatial choisie est la téléopération manuelle.

4.5.1 Protocole d'Exploration Spatiale

Une fois les nœuds de simulation et `slam_toolbox` lancés, le contrôle direct du robot différentiel s'effectue via un nœud d'interface clavier, publié sur le topic `/cmd_vel` :

```
ros2 run diff_robot teleop
```

Listing 4.4 – Activation du nœud de téléopération

Directives de pilotage pour un scan optimal :

1. **Vitesse de translation réduite** : Permet une haute densité de points LiDAR par mètre carré.
2. **Rotations douces** : Évite le phénomène de *motion blur* sur les balayages laser et permet au *Scan Matcher* de maintenir la cohérence de l'estimation de pose sans générer l'erreur fatale `Failed to compute odom pose`.
3. **Re-visite des zones (Loop Closure)** : Le pilotage inclut des passages répétés aux intersections clés du labyrinthe pour déclencher délibérément l'optimisation du graphe (Ceres).

4.5.2 Publication de la Carte et Correction Globale

À l'issue de l'optimisation par le solveur Ceres, SLAM Toolbox régénère la carte d'occupation sous forme d'une grille de cellules possédant trois états : libre, occupé (mur) ou inconnu.

En parallèle, le package calcule et publie dynamiquement la transformation `map` → `odom`. Cette TF est le résultat final du SLAM : elle représente l'écart mathématique cumulé entre l'odométrie mécanique (qui dérive physiquement) et la position réelle absolue du robot dans la carte globale.

4.5.3 Sauvegarde et Exportation de l'Occupancy Grid

La carte finale, générée par la fusion des scans positionnés et corrigés , est une "Grille d'Occupation" 2D où chaque cellule (`resolution: 0.05` mètres) est classée en trois états de probabilité : Libre, Occupée ou Inconnue.

Une fois l'exploration jugée satisfaisante sur l'interface RViz2, la carte globale est figée et extraite de la mémoire vive vers le disque dur à l'aide du `map_saver_cli` :

```
ros2 run nav2_map_server map_saver_cli -f ~/ros2_ws/src/maps/my_map
```

Listing 4.5 – Sauvegarde des artefacts de la carte

Cette commande génère automatiquement deux artefacts critiques pour la future pile de navigation (Nav2) :

— `my_map.pgm` : L'image matricielle en niveaux de gris représentant la topologie du monde physique.

— `my_map.yaml` : Le fichier de métadonnées contenant l'origine spatiale, les seuils d'occupation (`occupancy_threshold: 0.1`) et la résolution en mètres par pixel.

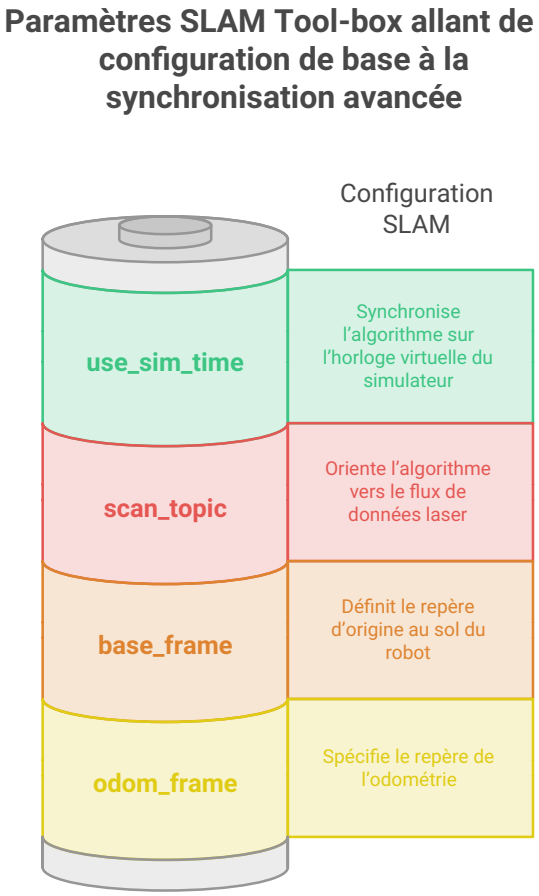


FIGURE 4.4 – Paramétrage de SLAM Toolbox

4.5.4 Visualisation et Gestion des Erreurs de Repères (TF)

Pour superviser l'évolution de la cartographie, l'équipe s'appuie sur l'interface **RViz2**. L'arborescence complète des transformations de coordonnées (*TF Tree*) orchestrée par le système respecte la hiérarchie standard :

`map` ⟶ `odom` ⟶ `base_footprint` ⟶ `laser_frame`

Une mauvaise configuration ou un retard de publication de ces repères provoque immédiatement le crash de l'algorithme avec l'erreur critique suivante dans le terminal : `Failed to compute odom pose`. Cette défaillance indique une rupture de synchronisation temporelle ou un nom de frame erroné entre le fichier URDF et le fichier YAML de configuration.

4.5.5 Configuration Manuelle de l'Interface RViz2

L'outil **RViz2** n'affiche aucune donnée par défaut au lancement. Pour visualiser l'invisible (les données brutes des nœuds ROS 2), l'équipe a dû ajouter et configurer minutieusement plusieurs modules d'affichage (*Displays*) dans le panneau latéral.

Le premier paramétrage a consisté à faire apparaître le squelette physique du robot en s'abonnant au topic `/robot_description` et en activant la visualisation de l'arbre des transformations (TF) pour vérifier le bon alignement des repères :

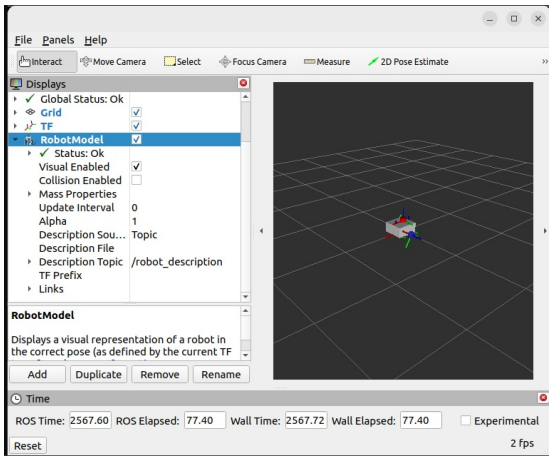


FIGURE 4.5 – Configuration du module RobotModel

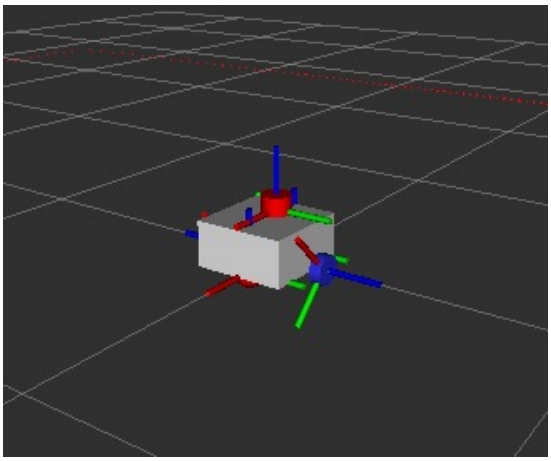


FIGURE 4.6 – Activation des repères spatiaux (TF)

Dans un second temps, il a fallu configurer les modules relatifs à la perception. Le *LaserScan* a été calibré sur le topic `/scan` pour afficher les points d'impact du LiDAR (en rouge), tandis que le module *Map* a été configuré pour lire la grille d'occupation générée par SLAM Toolbox sur le topic `/map`.

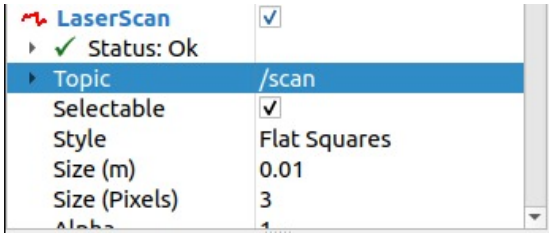


FIGURE 4.7 – Paramétrage du flux LaserScan

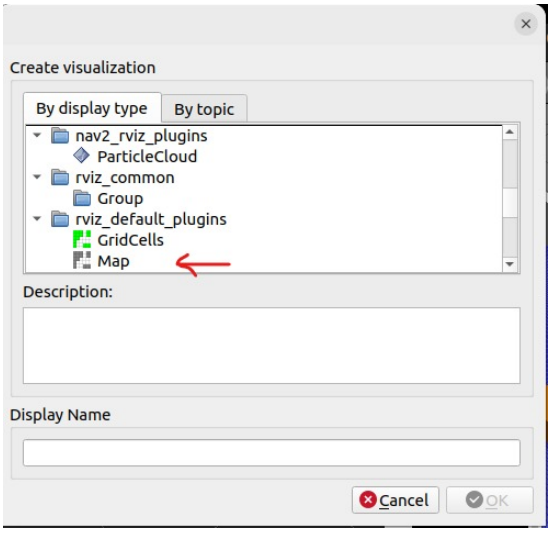


FIGURE 4.8 – Ajout de la grille d'occupation (Map)

Une fois l'ensemble de ces couches superposées et correctement abonnées à leurs topics respectifs, l'interface offre une supervision en temps réel d'une précision redoutable. Le rendu global (Figure 4.9) permet à l'équipe de corréler instantanément la position mécanique du robot, les obstacles détectés par le laser et la carte en cours de construction.

4.5.6 Génération Finale et Sauvegarde de la Grille d'Occupation

Au terme de la phase d'exploration, la carte générée prend la forme d'une **Grille d'Occupation 2D** stockée sur le topic `/map`. Cette grille est découpée en cellules possédant trois états discrets : libre, occupé (mur) ou inconnu.

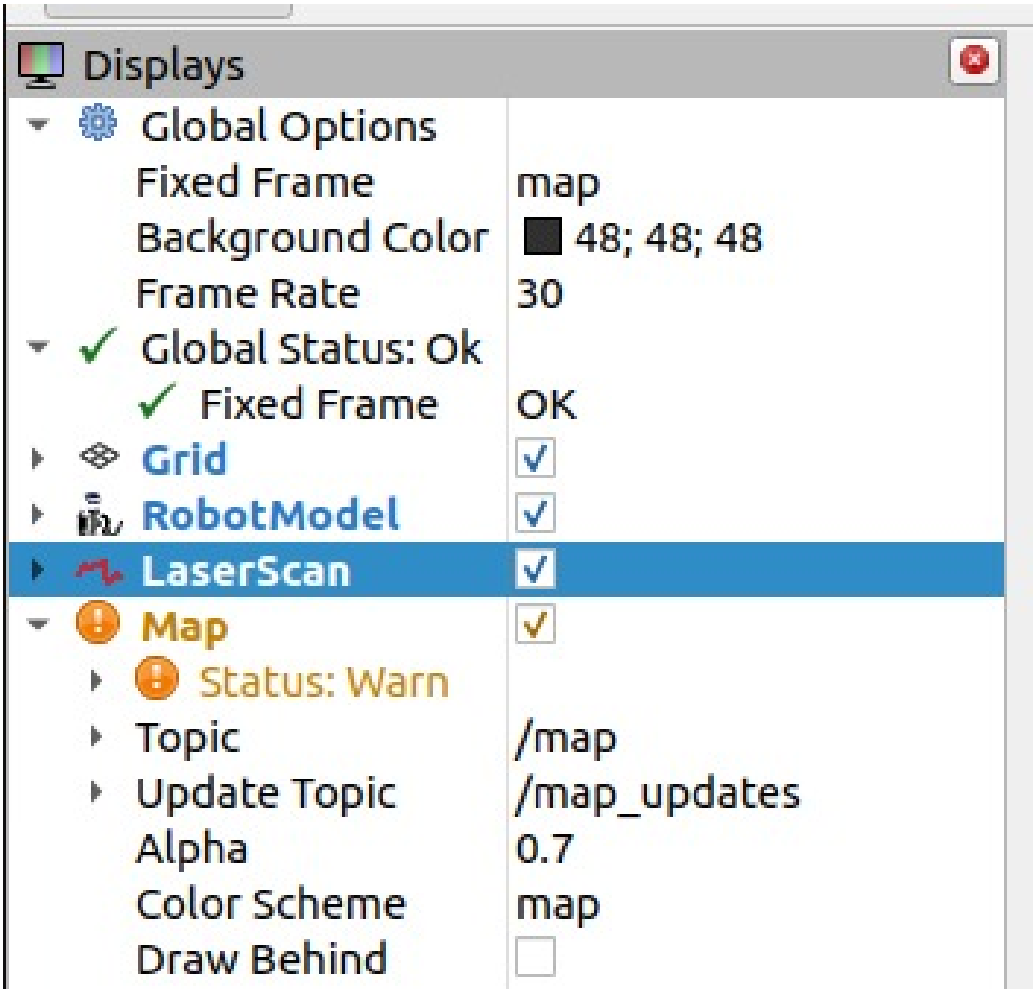


FIGURE 4.9 – Rendu global de l'environnement virtuel dans l'interface RViz2

Pour figer cet environnement avant la phase de navigation autonome, le pôle Perception exporte la carte à l'aide de l'outil de sauvegarde dédié de SLAM Toolbox, générant deux fichiers jumeaux dans le dossier `maps/` du projet :

→ `my_map.pgm` : L'image matricielle brute en niveaux de gris représentant la géométrie de la pièce.

→ `my_map.yaml` : Le fichier de métadonnées décrivant la résolution (ex : $0.05\,m$ par pixel), l'origine des coordonnées et les seuils de probabilité d'occupation.

CHAPITRE 5

NAVIGATION AUTONOME ET PLANIFICATION DE TRAJECTOIRES

5.1 Introduction au Framework Nav2

Une fois la carte de l'environnement générée par le SLAM, le robot possède une représentation spatiale de son monde. Cependant, cette carte n'est qu'une image morte. Pour s'y déplacer de manière autonome, éviter des obstacles imprévus et atteindre une coordonnée cible précise (*Goal Pose*), le système fait appel à la pile logicielle **Nav2** (Navigation 2).

Nav2 est une architecture modulaire complexe reposant sur des arbres de comportement (*Behavior Trees*). Son rôle est d'orchestrer une succession de calculs mathématiques pour transformer une coordonnée d'arrivée abstraite en commandes moteur concrètes.

5.2 Théorie des Cartes de Coût (Costmaps)

Pour que les algorithmes de planification fonctionnent, Nav2 ne lit pas la carte originale directement. Il la transforme en **Cartes de coût (Costmaps)**. Le principe est de diviser l'espace en une grille, où chaque cellule (pixel) se voit attribuer une valeur de "dangerosité" allant de 0 à 255.

- **Coût 0** : Espace libre, le robot peut rouler sans risque.
- **Coût 253** : Obstacle mortel (mur, objet).
- **Coût 254** : Obstacle mortel gonflé par l'empreinte du robot (collision garantie).
- **Coût 255** : Zone inconnue.

L'architecture s'appuie sur deux Costmaps distinctes, fonctionnant en parallèle :

5.2.1 La Global Costmap (Carte de coût globale)

Elle est statique et recouvre l'intégralité de l'environnement connu (issue du SLAM). Son rôle est de permettre au "cerveau lent" du robot de calculer la trajectoire optimale entre le point de départ et le point d'arrivée, en considérant que le monde ne bouge pas.

5.2.2 La Local Costmap (Carte de coût locale)

C'est une fenêtre glissante (ex : 3×3 mètres) centrée en permanence sur le robot. Contrairement à la carte globale, elle est **dynamique** et se met à jour à haute fréquence (souvent 5 Hz) grâce au flux en temps réel du LiDAR. Son rôle est de détecter les **obstacles éphémères** (une personne qui traverse, une chaise déplacée) pour permettre au robot d'adapter sa trajectoire locale sans recalculer tout le trajet global.

5.3 Configuration Stratégique des Paramètres (YAML)

Le succès de l'évitement d'obstacles repose sur le réglage fin des hyperparamètres géométriques du fichier `nav2_params.yaml`, une tâche critique réalisée lors du Sprint 2. Le listing 5.1 expose la configuration adoptée pour le projet GSEA2.

```yaml
# Fichier nav2_params.yaml
global_costmap:
  global_costmap:
    ros__parameters:
      robot_radius: 0.105
      resolution: 0.05
      use_sim_time: True
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        cost_scaling_factor: 3.0
        inflation_radius: 0.55
```

Listing 5.1 – Extrait de la configuration des couches d'inflation

5.3.1 Analyse de l'Empreinte : `robot_radius`

Le paramètre `robot_radius` (fixé à $0.105\,m$) est vital pour la survie du châssis. Nav2 traite par défaut le robot comme un simple "point" mathématique sans dimension. En déclarant ce rayon, nous forçons l'algorithme à faire une **vérification de collision circulaire**. Si le planificateur tente de faire passer le centre du robot à $10\,cm$ d'un mur, le calcul échouera et rejettera la trajectoire, car la carrosserie du robot frapperait l'obstacle.

5.3.2 La Bulle de Sécurité : `inflation_radius`

L'**Inflation Layer** (Couche de gonflement) est un algorithme qui propage artificiellement le coût d'un obstacle mortel vers l'extérieur de manière exponentielle décroissante. En fixant l'`inflation_radius` à $0.55\,m$, nous ordonnons au robot de maintenir une zone de confort autour des murs. Bien que le robot soit physiquement capable de frôler un mur à $12\,cm$, l'inflation le poussera à préférer une trajectoire évoluant au centre des couloirs.

5.3.3 Synchronisation Temporelle : `use_sim_time`

Le paramètre `use_sim_time: True` est une obligation absolue en robotique simulée. Il indique aux nœuds Nav2 et aux Costmaps de ne pas se fier à l'horloge physique de l'ordinateur, mais de s'abonner au topic `/clock` publié par Gazebo. Toute désynchronisation temporelle entre les capteurs virtuels et la carte entraînerait un échec de la transformée de repères (TF) et un arrêt du système.

5.4 La Planification : Global vs Local Planner

Avec les Costmaps correctement paramétrées, Nav2 délègue la gestion du mouvement à deux plugins complémentaires.

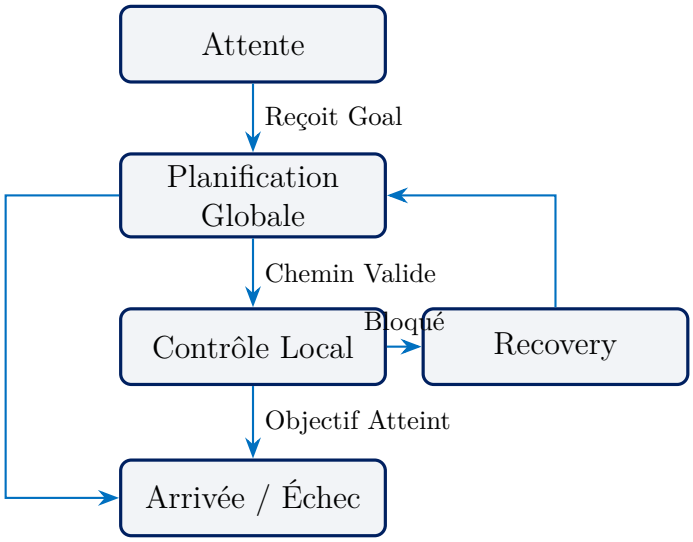


FIGURE 5.1 – Workflow détaillé de la stack de navigation Nav2

Le workflow Nav2 intègre des mécanismes de récupération (Recovery Behaviors) qui permettent au robot de se débloquer de manière autonome (ex : rotation sur place, recul) avant de relancer une nouvelle planification globale, assurant ainsi une haute disponibilité du système.

5.4.1 Le Planificateur Global (`NavfnPlanner`)

C'est le stratège du système. Son unique mission est de tracer une ligne brisée (le chemin le plus court et le moins dangereux) du robot vers la cible. Suite aux tests du Sprint 3, l'équipe a forcé l'utilisation de l'algorithme d'exploration **A*** (A-Star) en configurant le paramètre `use_astar: true`, ce dernier s'avérant nettement plus performant et rapide que l'algorithme de **Dijkstra** classique dans des environnements complexes fermés. L'option `allow_unknown: true` a également été activée pour permettre au robot de tracer des trajectoires traversant des zones grisées non totalement explorées.

5.4.2 Le Contrôleur Local (`DWBLocalPlanner`)

C'est le pilote du système. Il lit la trajectoire dessinée par le planificateur global et regarde la *Local Costmap*. En utilisant l'approche **Dynamic Window Approach**

(DWB), il génère des dizaines de trajectoires courbes possibles (arcs de cercle) dans la seconde qui suit. Il attribue une note à chaque trajectoire selon trois critères :

1. **Path Distance** : Est-ce que cet arc me rapproche du chemin global ?
2. **Goal Distance** : Est-ce que cet arc me rapproche de la cible finale ?
3. **Obstacle Cost** : Est-ce que cet arc frôle la zone d'inflation d'un obstacle ?

La trajectoire obtenant le meilleur score global est traduite en commandes de vitesses (`cmd_vel`) et envoyée directement aux moteurs différentiels dans Gazebo, achevant ainsi la boucle d'autonomie.

5.5 Localisation Probabiliste : L'Algorithme AMCL

Avoir une carte statique ne suffit pas à naviguer : le robot doit d'abord savoir où il se trouve sur cette carte. C'est le rôle du système de localisation de Nav2, qui s'appuie sur le nœud **AMCL** (*Adaptive Monte Carlo Localization*).

AMCL est un algorithme probabiliste qui génère un "nuage de particules" autour de la position supposée du robot. Chaque particule représente une hypothèse de position (x, y, θ). Au fur et à mesure que le robot avance et que son LiDAR scanne l'environnement, l'algorithme compare ces scans avec la carte connue. Les particules incompatibles avec les obstacles sont détruites, tandis que les particules cohérentes se multiplient, convergeant ainsi vers la position réelle du robot.

5.5.1 La Calibration de la Pose Initiale (2D Pose Estimate)

Comme l'a souligné le pôle Navigation lors de ses tests du Sprint 2 : *« La calibration de la pose initiale via AMCL est l'étape charnière entre la cartographie et la navigation : sans elle, le robot dispose d'une carte mais est incapable de l'utiliser. »*

Au lancement de Nav2, le robot est mathématiquement "perdu" (le nuage de particules est dispersé sur toute la carte). L'opérateur doit lui fournir une estimation initiale manuelle via l'outil **2D Pose Estimate** de RViz2. Une fois cette pose humaine injectée, les repères `map` et `odom` se synchronisent fermement, et la navigation devient possible.

5.6 Exécution d'une Requête de Navigation (Actions ROS 2)

Contrairement aux *Topics* (flux continus unidirectionnels) ou aux *Services* (requêtes instantanées), la navigation vers un point cible (*Waypoint*) utilise le mécanisme des **Actions ROS 2**. Une Action permet d'envoyer un objectif long à atteindre, tout en recevant un retour d'état (*Feedback*) continu pendant le trajet, avec la possibilité de l'annuler en cours de route.

Le pôle a validé l'intégrité de la chaîne de navigation en envoyant une requête directe au serveur d'action `/navigate_to_pose` via l'interface en ligne de commande (CLI) :

```
# Envoi d'un point cible aux coordonnees X=1.0m, Y=0.5m par rapport a la map
$ ros2 action send_goal /navigate_to_pose \
    nav2_msgs/action/NavigateToPose \
    '{pose: {header: {frame_id: map}, pose: {position: {x: 1.0, y: 0.5}}}}'
```

Listing 5.2 – Envoi manuel d'un objectif de navigation (Waypoint) via le terminal

LEÇON TIRÉE DE L'INGÉNIERIE DU PROJET

En robotique autonome, la question n'est pas seulement *"où est le robot ?"* mais *"comment le robot sait-il où il est ?"*. La réponse combine des capteurs fiables (`/scan`, `/odom`), un algorithme probabiliste robuste (AMCL) et une estimation initiale humaine. Ces trois éléments forment le socle incontournable de toute autonomie réelle.

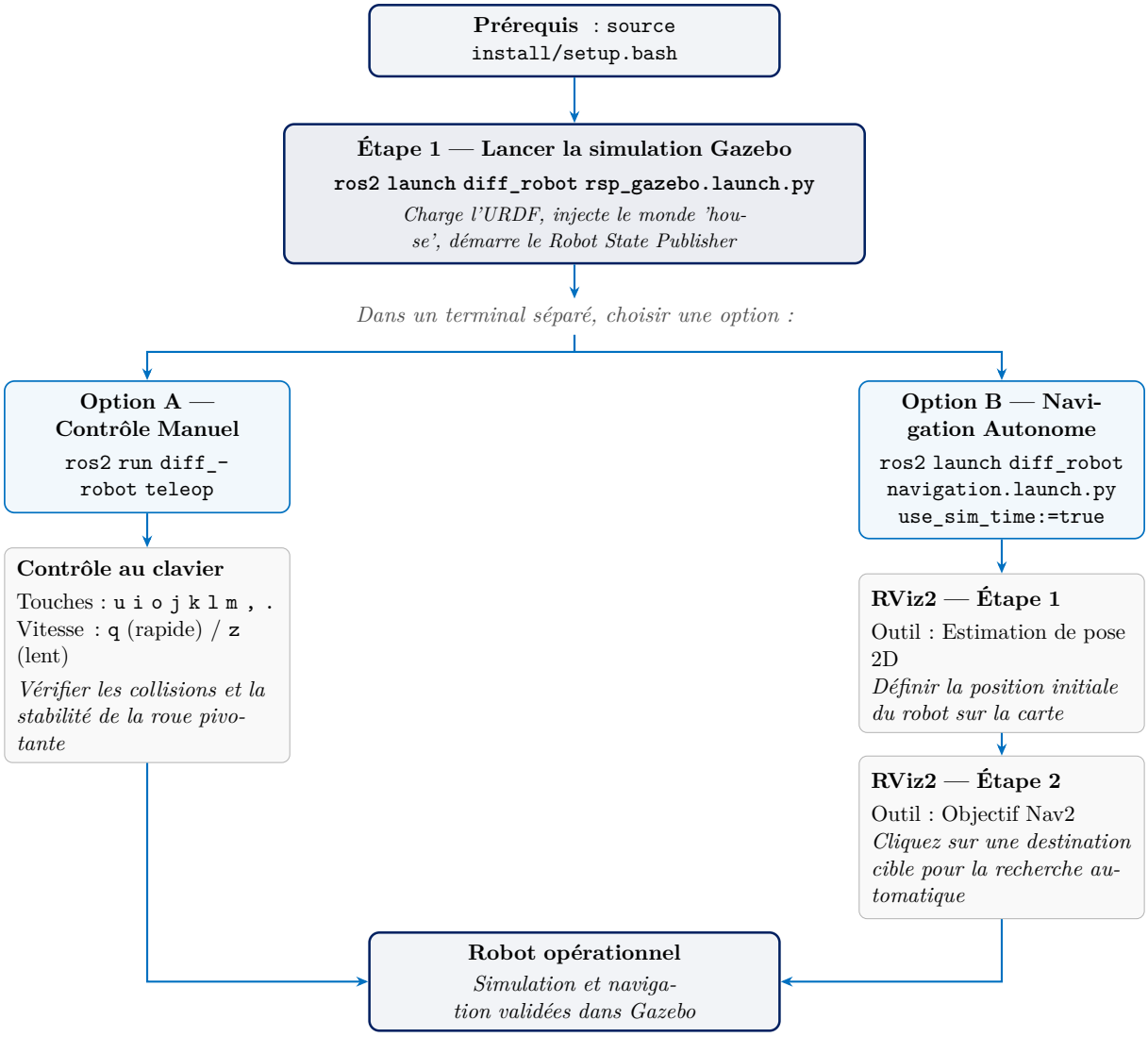


FIGURE 5.2 – Flux de travail : Test et simulation du robot

5.7 Validation et Tests d'Évitement d'Obstacles

L'aboutissement du Sprint 3 s'est traduit par une campagne de tests intensifs dans l'environnement virtuel `diff_robot.world`. Avant de valider le comportement du robot,

le pôle Contrôle a procédé à un ajustement critique des contraintes cinématiques au sein du `DWBLocalPlanner` :

```
max_vel_x: 0.22      # Vitesse maximale en marche avant (m/s)
min_vel_x: -0.22     # Vitesse maximale en marche arriere (m/s)
max_vel_theta: 2.75  # Vitesse de rotation maximale (rad/s)
```

Listing 5.3 – Ajustement des contraintes de vélocité du planificateur local

Ce bridage de la vélocité (`max_vel_x: 0.22 m/s`) garantit que le robot ne roule pas plus vite que la fréquence de rafraîchissement de sa Costmap locale (5 Hz), lui laissant le temps de freiner face à un obstacle imprévu.

5.7.1 Scénarios de Simulation Validés et Analyse Visuelle

La validation expérimentale du système s'est appuyée sur une série de tests rigoureux visant à éprouver la robustesse de la stack Nav2 face à la géométrie de l'environnement virtuel.

1. Génération et Évolution de l'Environnement 3D (Gazebo)

La première phase de validation a consisté à vérifier la stabilité du chargement des structures du labyrinthe. La Figure 5.5 met en opposition le monde d'essai initial comportant des obstacles isolés et le monde final complet utilisé pour la validation de la thèse, intégrant une topologie multi-pièces réaliste.

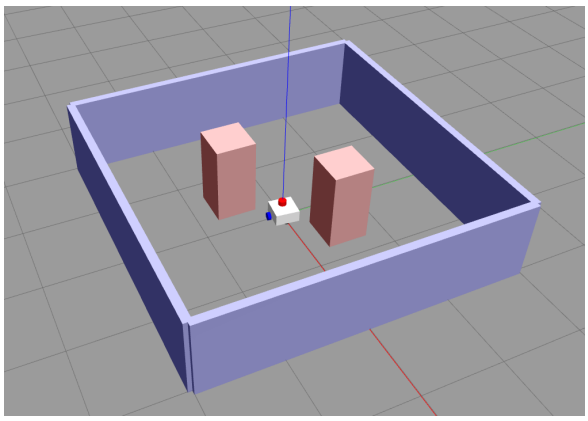


FIGURE 5.3 – Monde de démonstration initial

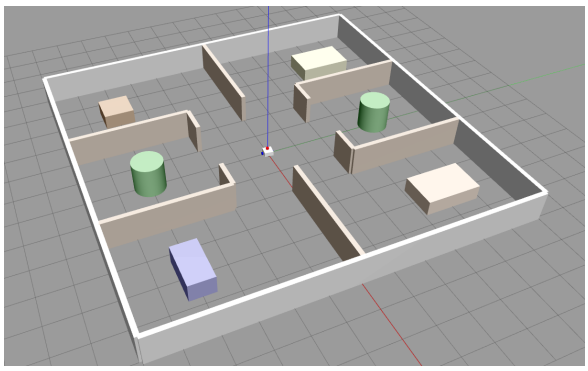


FIGURE 5.4 – Monde labyrinthique de validation finale

FIGURE 5.5 – Évolution des environnements de simulation 3D sous Gazebo

2. Supervision Synchrone de la Planification

Lorsqu'un objectif de navigation (*Waypoint*) est soumis au robot, la corrélation entre l'espace physique simulé et l'espace logique calculé doit être parfaite. Les Figures 5.6 et 5.7 illustrent cette synchronisation : pendant que Gazebo gère l'application des forces sur les articulations physiques, RViz2 affiche la projection de la trajectoire optimale calculée par le planificateur global A^*.

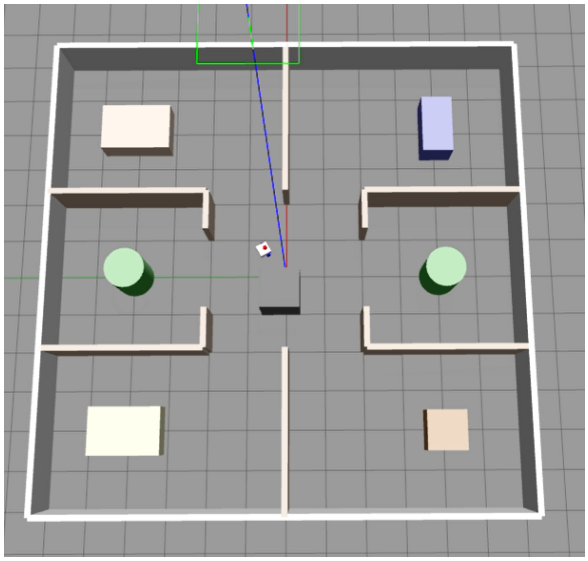


FIGURE 5.6 – Vue physique du robot dans Gazebo

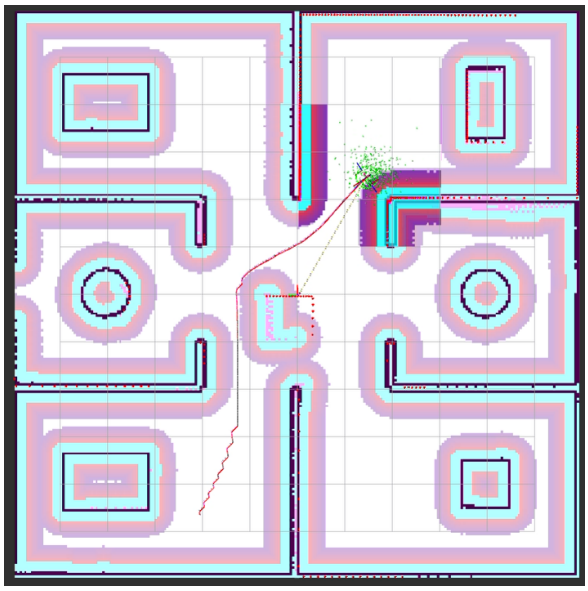


FIGURE 5.7 – Visualisation du chemin global dans RViz2

FIGURE 5.8 – Corrélation synchrone de la navigation : Physique vs Logique

3. Comportement du Robot Face aux Obstacles Complexes

Les scénarios de test les plus critiques concernaient le franchissement des zones étroites et le contournement de formes non convexes. La Figure 5.9 expose un focus sur l'empreinte géométrique du robot au sein de la grille, tandis que la Figure 5.12 démontre la réussite du calcul d'évitement dynamique.

Le planificateur local `DWBLocalPlanner` parvient à infléchir la trajectoire en temps réel pour maintenir le centre du robot en dehors des zones d'inflation critiques (pixels colorés), validant ainsi l'efficacité des réglages de sécurité programmés.

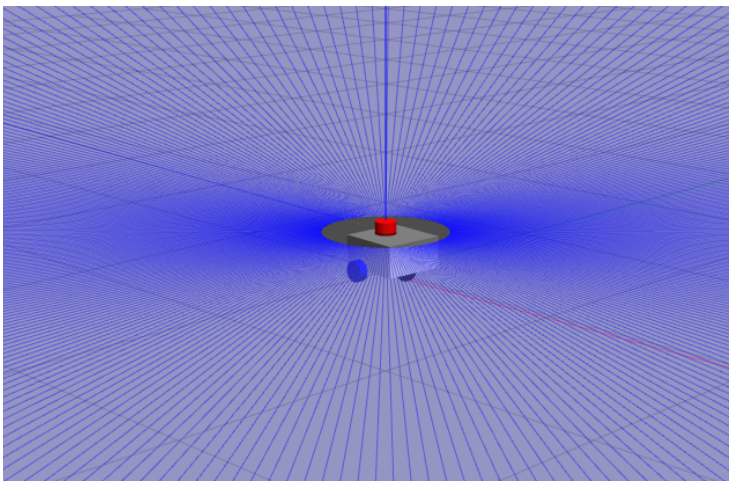


FIGURE 5.9 – Analyse de l'empreinte circulaire du modèle de base sous RViz2

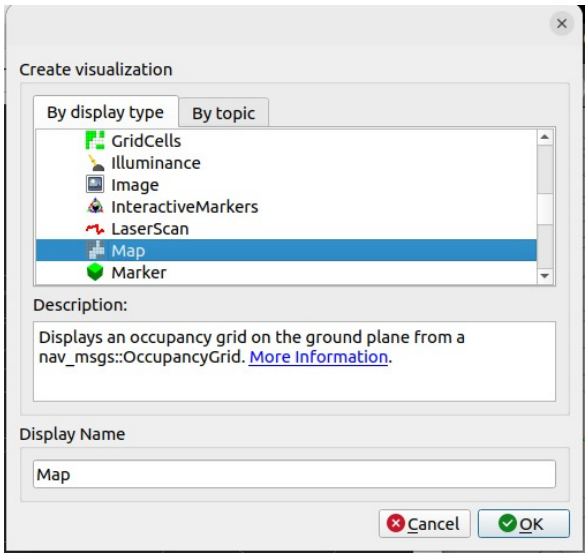


FIGURE 5.10 – Planification locale en couloir étroit

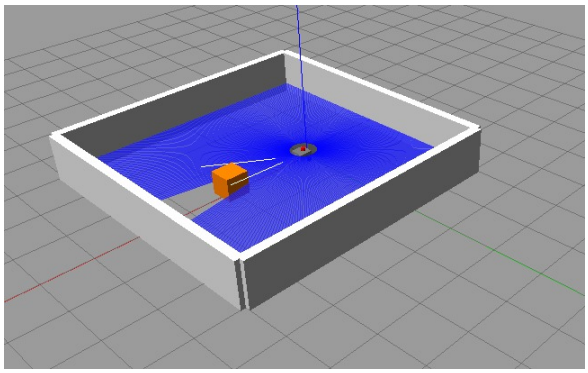


FIGURE 5.11 – Contournement réussi d'un obstacle non convexe

FIGURE 5.12 – Comportement dynamique du nuage de particules AMCL et de la trajectoire locale

5.7.2 Bilan de Synthèse du Pôle Navigation

Au terme de ces campagnes de mesures visuelles, l'ensemble des indicateurs de performance logicielle est validé. Le robot se localise sans dérive majeure grâce à la convergence rapide du filtre AMCL, et l'asservissement des moteurs différentiels répond aux exigences cinématiques de la trajectoire locale, assurant une navigation autonome fluide et sécurisée d'un bout à l'autre du labyrinthe virtuel.

CHAPITRE 6

CONCLUSION GÉNÉRALE ET PERSPECTIVES

6.1 Bilan Technique et Synthèse du Projet

La réalisation de ce projet de fin d'études au sein de la formation **GSEA2** constitue une expérience d'ingénierie particulièrement enrichissante. L'objectif initial, qui consistait à concevoir, modéliser et programmer un **Robot Mobile Autonome** de A à Z en environnement virtuel, a été pleinement atteint à travers une démarche rigoureuse et structurée.

Au cours du **Sprint 1**, notre pôle s'est approprié les concepts cinématiques du déplacement différentiel. Nous avons traduit ces modèles physiques en descriptions géométriques et inertielles standardisées via le langage **URDF/Xacro**. Le développement réussi du nœud de téléopération asynchrone `robot_teleop` a validé la réactivité des actionneurs virtuels et la stabilité du châssis au sein du moteur physique tridimensionnel **Gazebo**.

Le **Sprint 2** a marqué le passage de la commande manuelle à l'intelligence artificielle distribuée. En exploitant la puissance du middleware **ROS 2 Humble**, nous avons interconnecté de manière synchrone les flux sensoriels bruts du **LiDAR 2D** (topic `/scan`) et de la centrale inertielle **IMU** (topic `/imu`). L'implémentation de la suite **SLAM Toolbox** a permis de résoudre l'équation de la cartographie simultanée.

Enfin, l'intégration fine du framework **Nav2** et le paramétrage stratégique des couches d'inflation des **Costmaps** (`robot_radius` à $0.105\,m$ et `inflation_radius` à $0.55\,m$) ont fourni au robot la capacité de planifier des trajectoires optimales via l'algorithme A^* et d'éviter dynamiquement les obstacles imprévus.

6.2 Difficultés Résolues et Compétences Acquises

Un projet de cette envergure ne s'accomplit pas sans heurts. L'équipe a dû faire face à des contraintes techniques majeures, notamment liées au développement au sein d'une **Machine Virtuelle (VirtualBox)**. Le contournement du crash de rendu de Gazebo (`code 255`) par l'injection de la variable logicielle `LIBGL_ALWAYS_SOFTWARE=1` a constitué une leçon précieuse de gestion de l'infrastructure système sous Linux. De même, la

parallélisation du code à cinq développeurs nous a poussés à standardiser un workflow **Git/GitHub** rigoureux pour surmonter les conflits de fusion.

Sur le plan personnel et professionnel, ce projet nous a permis d'acquérir des compétences de pointe très recherchées dans l'industrie de l'industrie 4.0 :

- Maîtrise de l'environnement **Linux Ubuntu** et du scripting en ligne de commande.
- Programmation orientée objet en **Python** appliquée aux nœuds et architectures robotiques.
- Compréhension approfondie des architectures réseaux décentralisées et du temps réel via le middleware **DDS**.
- Gestion de projet industrielle selon la méthodologie **Agile / Scrum**.

6.3 Perspectives de Développement Futur

Bien que le système actuel soit totalement fonctionnel en simulation, ce travail pose les jalons de développements futurs passionnants qui pourraient élever ce projet au rang de produit industriel :

6.3.1 Migration vers une Plateforme Physique Réelle

La suite logique de cette thèse virtuelle est le déploiement de notre pile logicielle (stack) sur un véritable robot physique à entraînement différentiel (par exemple, un châssis équipé d'un ordinateur monocarte *Raspberry Pi 4/5* ou *NVIDIA Jetson Nano*, de moteurs pas-à-pas avec encodeurs, d'un vrai LiDAR RPLIDAR A1/A2 et d'une centrale MPU6050). Cela permettrait de confronter nos filtres d'odométrie aux bruits physiques et au glissement mécanique du monde réel.

6.3.2 Intégration de la Vision par Ordinateur (Visual SLAM)

Remplacer ou compléter le LiDAR 2D par une caméra de profondeur 3D (type *Intel RealSense*) permettrait d'implémenter des algorithmes de SLAM visuel (V-SLAM) comme *ORB-SLAM3* ou *RTAB-Map*. Le robot ne se contenterait plus de voir des lignes de murs en 2D, mais pourrait reconnaître des objets texturés en 3D, ouvrant la voie à la classification sémantique de son environnement.

6.3.3 Évitement d'Obstacles par Apprentissage par Renforcement (IA)

Bien que le planificateur local `DWBLocalPlanner` soit très performant, ses trajectoires restent purement géométriques. Une perspective d'avenir majeure serait d'entraîner le robot à l'évitement d'obstacles dynamiques complexes en couplant ROS 2 avec des frameworks d'Intelligence Artificielle comme *TensorFlow* ou *PyTorch*, en utilisant l'apprentissage par renforcement profond (Deep Reinforcement Learning).

6.3.4 Robotique Collaborative (Flotte de Robots)

Grâce aux capacités de découverte automatique du protocole DDS de ROS 2, une extension naturelle serait d'introduire un deuxième robot dans la simulation pour réali-

ser de la cartographie collaborative distribuée (Multi-Robot SLAM), où plusieurs unités partagent et fusionnent leurs cartes en temps réel pour explorer un labyrinthe deux fois plus vite.

ANNEXES

ANNEXE A

GUIDE DE REPRODUCTIBILITÉ ET DÉPÔT GITHUB

L'intégralité du code source, des modèles 3D (URDF/Xacro), des scripts de lancement et des fichiers de configuration développés durant les Sprints 1, 2 et 3 est versionnée et disponible en accès libre sur notre dépôt GitHub.

Dépôt Officiel du Projet avec des vidéos de simulations

Cliquez ici pour accéder au code source

Procédure d'installation Rapide

Pour déployer ce projet sur une machine fraîchement installée sous Ubuntu 22.04 et ROS 2 Humble, exécutez les commandes suivantes dans un terminal :

```
# 1. Mise a jour du systeme et installation des dependances
$ sudo apt update && sudo apt upgrade -y
$ sudo apt install ros-humble-navigation2 ros-humble-nav2-bringup ros-
    humble-slam-toolbox -y

# 2. Creation de l'espace de travail et clonage
$ mkdir -p ~/ros2_ws/src && cd ~/ros2_ws/src
$ git clone https://github.com/VOTRE_NOM_D_UTILISATEUR/autonomous-mobile-
    robot-ros2.git

# 3. Compilation du projet
$ cd ~/ros2_ws
$ colcon build --symlink-install
$ source install/setup.bash
```

Listing A.1 – Commandes d'installation du projet

ANNEXE B

EXTRAITS DE CODE : CONFIGURATION NAV2

Ce fichier YAML contient les réglages fins ajustés par le pôle Navigation lors du Sprint 3. Il définit notamment les dimensions du robot et les contraintes cinématiques de l'algorithme DWB.

```yaml
amcl:
  ros__parameters:
    alpha1: 0.2
    alpha2: 0.2
    alpha3: 0.2
    alpha4: 0.2
    alpha5: 0.2
    base_frame_id: base_link
    beam_skip_distance: 0.5
    beam_skip_error_threshold: 0.9
    beam_skip_threshold: 0.3
    do_beamskip: false
    global_frame_id: map
    lambda_short: 0.1
    laser_likelihood_max_dist: 2.0
    laser_max_range: 100.0
    laser_min_range: 0.3
    laser_model_type: likelihood_field
    max_beams: 60
    max_particles: 2000
    min_particles: 500
    odom_frame_id: odom
    pf_err: 0.05
    pf_z: 0.99
    recovery_alpha_fast: 0.0
    recovery_alpha_slow: 0.0
    resample_interval: 1
```

Listing B.1 – Extrait du fichier `nav2_params.yaml` (DWB Local Planner)

ANNEXE C

SCRIPT DE LANCEMENT PRINCIPAL (LAUNCH FILE)

Le script Python ci-dessous est le point d'entrée principal de notre système. Il est responsable du chargement du fichier Xacro, de l'instanciation de l'état du robot et de la connexion avec le simulateur Gazebo.

```python
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
import xacro

# basic launch file to launch the robot_state_publisher with the core_robot
    .xacro.urdf file

def generate_launch_description():
    # package name and urdf file sub path
    pkg_name = "diff_robot"
    subpath_to_description = "description/core_robot.xacro.urdf"
    # full path of the urdf file
    full_path_to_description = os.path.join(get_package_share_directory(
    pkg_name) , subpath_to_description)
    # we need to xacro.urdf > .urdf

    world_file_name = "diff_robot.world"
    world_path = os.path.join(get_package_share_directory(pkg_name), 'world
    ', world_file_name)

    xacro_description_file = xacro.process_file(full_path_to_description).
    toxml()
    # configure the robot state publisher node
    rspLaunchFile = Node(
        package = 'robot_state_publisher',
        executable= 'robot_state_publisher',
        output = 'screen',
        parameters= [{'robot_description': xacro_description_file ,
```

```
                        'use_sim_time' : True}]
    )
    # configure the gazebo node and the spawn entity node
    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
            get_package_share_directory('gazebo_ros'), 'launch'), '/gazebo.
    launch.py']),
            launch_arguments={'world': world_path}.items()
        )




    # pour spawn the entity
    spawn_entity = Node(package='gazebo_ros', executable='spawn_entity.py',
                    arguments=['-topic', 'robot_description',
                               '-entity', 'my_bot'],
                    output='screen')


    return LaunchDescription([
        gazebo ,
        spawn_entity ,
        rspLaunchFile
    ])
```

Listing C.1 – Extrait de `rsp_gazebo.launch.py`

ANNEXE D

SCRIPT DE LANCEMENT DE LA NAVIGATION (NAV2)

Ce script Python (`navigation.launch.py`) est chargé d'initialiser l'ensemble de la stack d'autonomie. Il récupère la carte statique générée par le pôle Perception (`my_map.yaml`), applique les contraintes cinématiques et d'évitement d'obstacles (`nav2_params.yaml`), et lance simultanément l'interface de supervision RViz2 avec une configuration visuelle prédéfinie.

```python
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription, DeclareLaunchArgument
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    pkg_name = 'diff_robot'
    pkg_share = get_package_share_directory(pkg_name)

    # Chemins absolus vers les fichiers de configuration
    map_file = os.path.join(pkg_share, 'maps', 'my_map.yaml')
    params_file = os.path.join(pkg_share, 'config', 'nav2_params.yaml')
    rviz_config_file = os.path.join(pkg_share, 'config', 'navigation.rviz')

    use_sim_time = LaunchConfiguration('use_sim_time', default='true')
    nav2_launch_dir = os.path.join(get_package_share_directory('
    nav2_bringup'), 'launch')

    # Inclusion de la stack Nav2 standard
    nav2_cmd = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(nav2_launch_dir, '
    bringup_launch.py')),
        launch_arguments={
            'map': map_file,
            'params_file': params_file,
            'use_sim_time': use_sim_time,
        }.items()
```

```
    )

    # Lancement preconfigure de RViz2
    rviz_node = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        arguments=['-d', rviz_config_file],
        parameters=[{'use_sim_time': use_sim_time}],
        output='screen'
    )

    return LaunchDescription([
        DeclareLaunchArgument(
            'use_sim_time',
            default_value='true',
            description='Use simulation (Gazebo) clock if true'),
        nav2_cmd,
        rviz_node
    ])
```

Listing D.1 – Script d'automatisation `navigation.launch.py`

RÉFÉRENCES BIBLIOGRAPHIQUES

Books

[1] Documentation officielle ROS 2. *ROS 2 Humble Hawksbill*. Disponible sur : https://docs.ros.org/en/humble/index.html

Articles

[2] Macenski, S., et al. *The Nav2 Project*. Framework de navigation autonome. Disponible sur : https://navigation.ros.org/

[3] Open Robotics Discourse. *UrdfArchitect : An AI-powered visual editor to build robot models without manual XML coding*. Disponible sur : https://discourse.openrobotics.org/t/urdfarchitect-a-ai-powered-visual-editor-to-build-robot-models-without-manual-xml-51484/2 (Consulté le 29 juin 2026).

Chapitre de livres

[4] ROBOTIS. *TurtleBot3 e-Manual*. Documentation technique du robot différentiel. Disponible sur : https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/

Cours et Ouvrages Académiques

[5] Stasse, O. *Robot Operating System : Introduction*. Polycopié de cours, LAAS-CNRS, 2022.

Autres

[6] Middleware : logiciel. Source : *circleci*. Disponible sur : https://circleci.com/blog/what-is-middleware/

[7] Iun extrait du code Xacro de notre robot décrivant son châssis principal. Source extraite de : *ROS / Documentation officielle*. Disponible sur : http://www.ros.org/wiki/xacro

[8] Illustration d'un capteur LiDAR. Source : *youyeetoo*. Disponible sur : https://youyeetoo.com/products/ydlidar-x2l

[9] Robotry. *Robotics and ROS2 : Easy Guide FREE*. Disponible sur : https://robotry.in/course/robotics-and-ros2-easy-guide-free/ .

[10] Illustration d'un capteur IMU. Source : *photo sur internet*. Disponible sur : https://metisengineering.com/wp-content/uploads/2024/07/CANIMUGEN1JWPF.png

[11] Cadden. *Lidar : comment fonctionne la technologie ?*. Disponible sur : https://www.cadden.fr/fonctionnement-technologie-lidar/# .

[12] D-Robotics. *URDF Studio*. Interface en ligne. Disponible sur : https://urdf.d-robotics.cc/ .